

Ingenieria del Software

Tema 4. Diseño

TABLA DE CONTENIDOS

01 **Arquitectura**

02 **Diseño**

03 **Interface y Experiencia de Usuario (UX/UI)**

TABLA DE CONTENIDOS

01

Arquitectura

02

Diseño

03

Interface y Experiencia de Usuario (UX/UI)

Arquitectura

¿Qué es la Arquitectura del Software?

La arquitectura del software establece las bases, los pilares, la estructura, sobre la cual reposará el resto de componentes. Estos “**cimientos**” deben ayudar a minimizar el esfuerzo de desarrollo, despliegue y mantenimiento del producto software.

“La arquitectura del software son las decisiones de diseño significativas que dan forma a un sistema, donde la importancia se mide por el costo del cambio”

Souza y Cameron, 1998

¿Qué es la Arquitectura del Software?

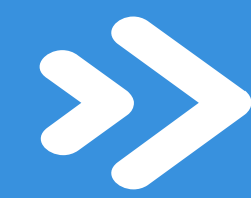
La arquitectura del sistema se puede estudiar bajo los siguientes prismas:

- » **Arquitectura de Integración**
- » **Arquitectura de Componentes**
- » **Arquitectura de Desarrollo**

¿Qué es la Arquitectura del Software?

El objetivo del diseño del software es concretar y aterrizar los componentes especificados en el análisis.

- **Arquitectura de Integración**
- **Arquitectura de Componentes**
- **Arquitectura de Desarrollo**

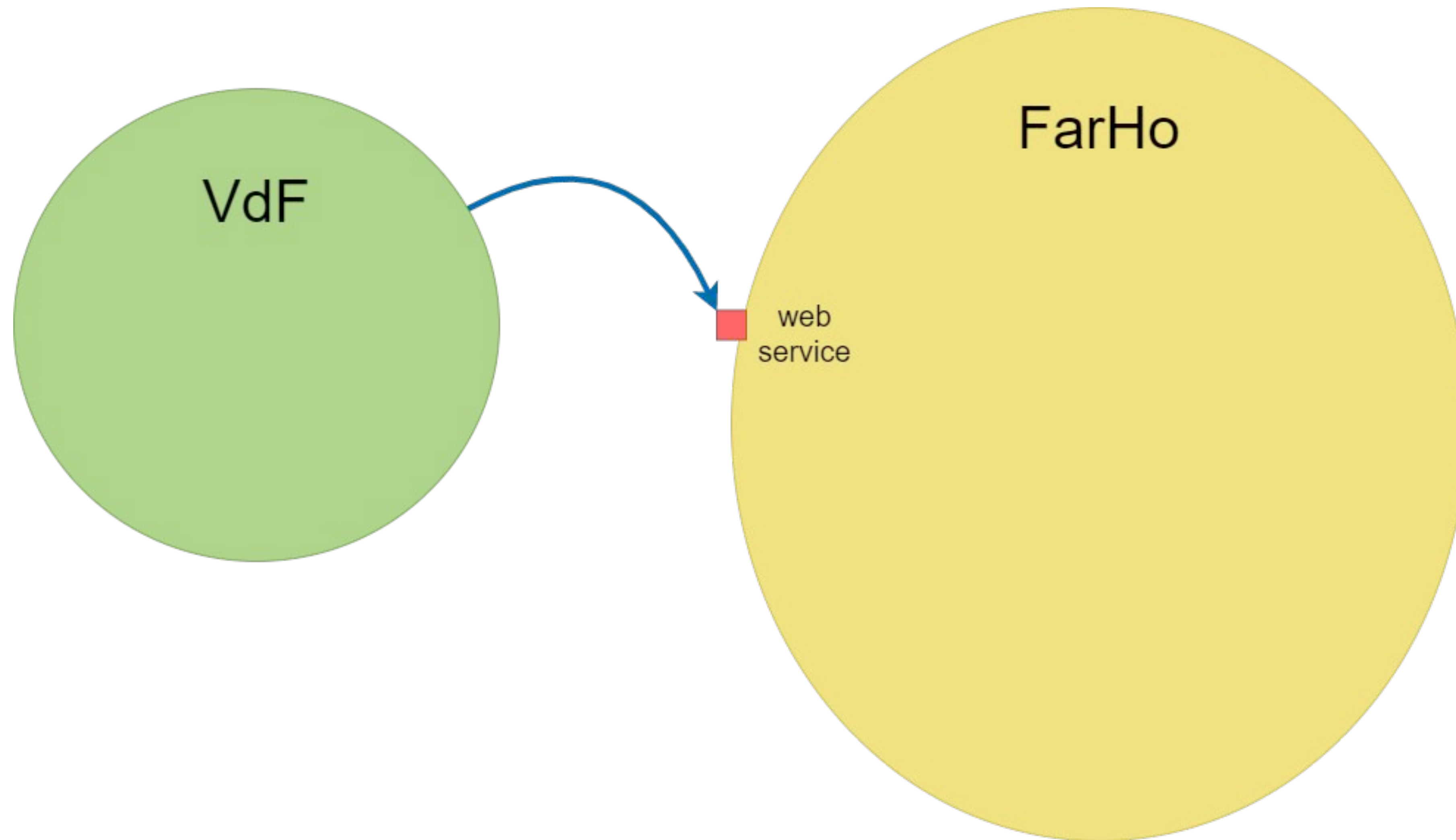


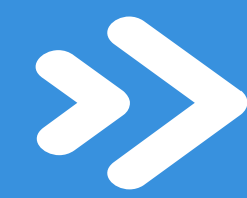
Arquitectura de Integración

Los sistemas de información deben integrarse entre sí para intercambiar información, bien por cuestiones funcionales, de disponibilidad, etc.

Estas integraciones pueden realizarse de diferentes maneras, siendo algunas de ellas más o menos adecuadas según las necesidades concretas de cada caso.

La arquitectura de integración pone el foco en cómo los productos software intercambian información identificando los pilares básicos de comunicación

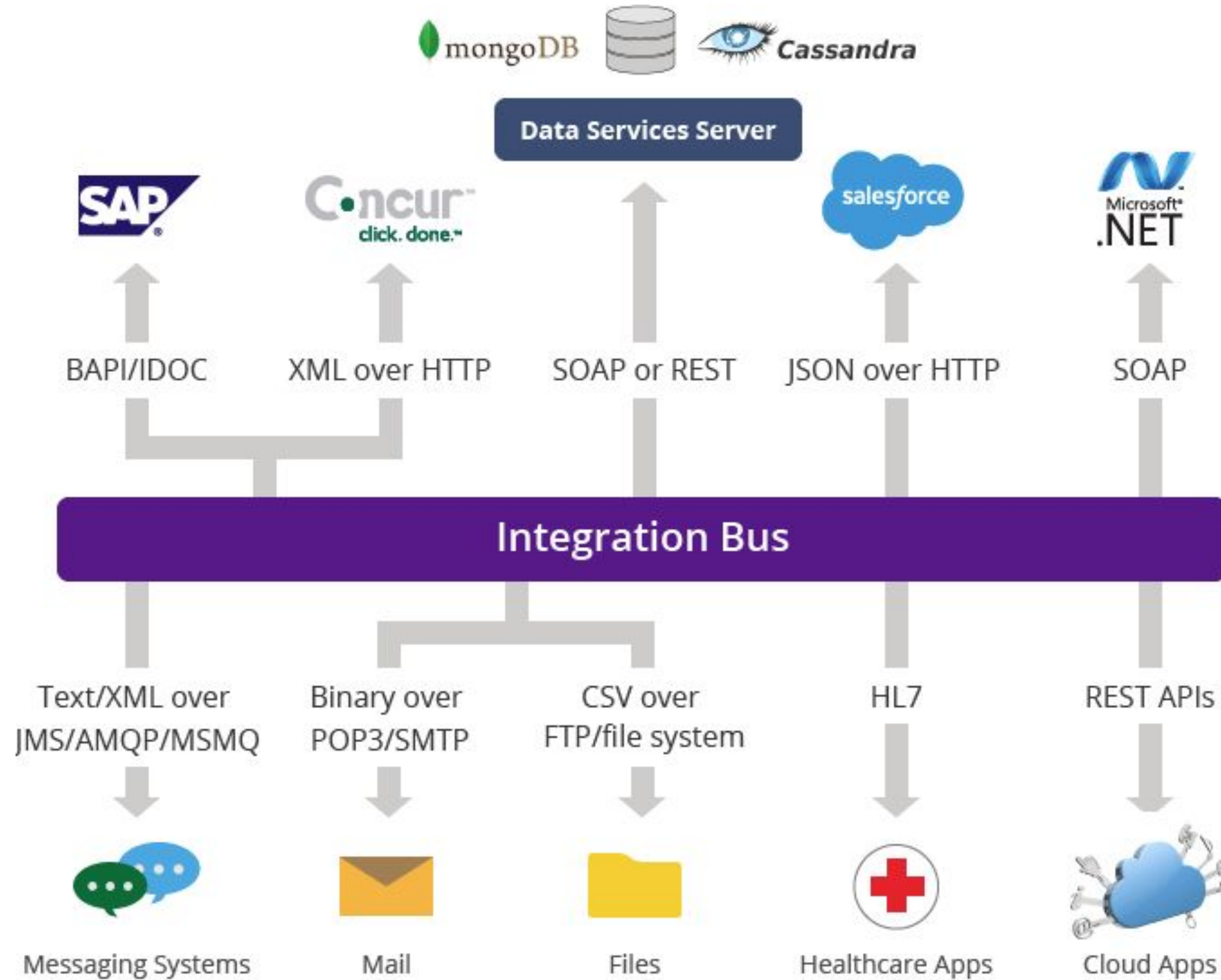




Las formas más habituales de intercambio de datos entre productos son:

- ❑ **Acceso directo a la base de datos.** Un producto obtiene datos de otro accediendo directamente a las tablas donde el otro producto almacena la información. En este escenario, la ausencia de contrato de servicio y la falta de lógica de negocio impone alto acoplamiento lógico (y tecnológico) y baja reusabilidad.
- ❑ **Intercambio de ficheros de texto.** Los productos generan ficheros de texto con los datos que luego intercambian. No es posible una comunicación síncrona de los datos. El consumidor accede a una “foto” de la información expuesta por el propietario de los datos.
- ❑ **Invocación a Servicios Web.** Un servicio web es un puerto de comunicaciones ofrecido por un producto que, utilizando como capa de transporte el protocolo HTTP, ofrece datos en un formato estándar (XML o JSON) a través de mensajes normalizados (SOAP o REST). Al procesar el mensaje recibido el producto propietario de los datos puede ejecutar ciertas reglas de negocio adaptando de esta forma la información. Los web services son el mecanismo habitual de intercambio de información entre productos software.

>> Arquitectura de Integración

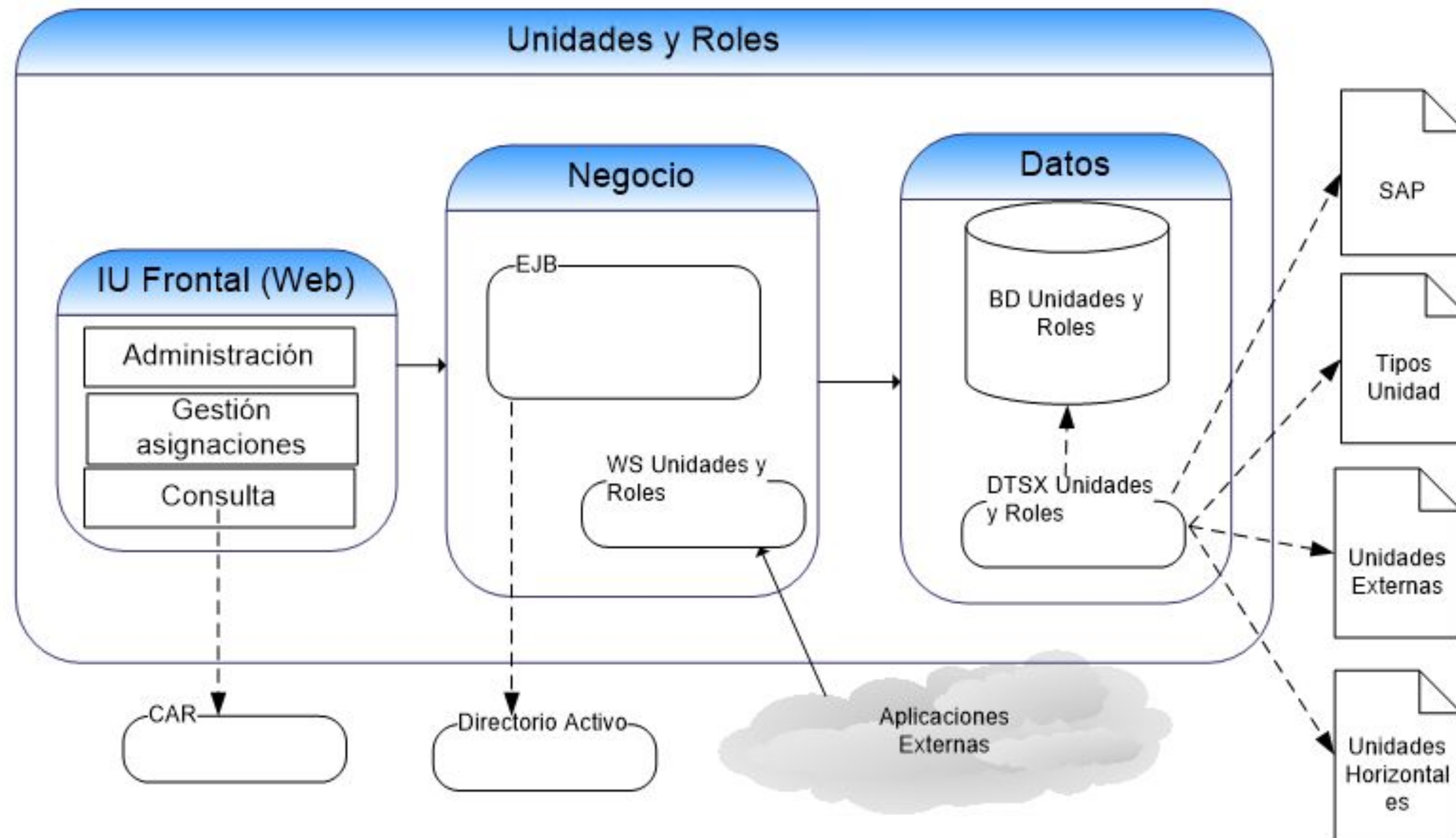


¿Qué es la Arquitectura del Software?

El objetivo del diseño del software es concretar y aterrizar los componentes especificados en el análisis.

- **Arquitectura de Integración**
- **Arquitectura de Componentes**
- **Arquitectura de Desarrollo**

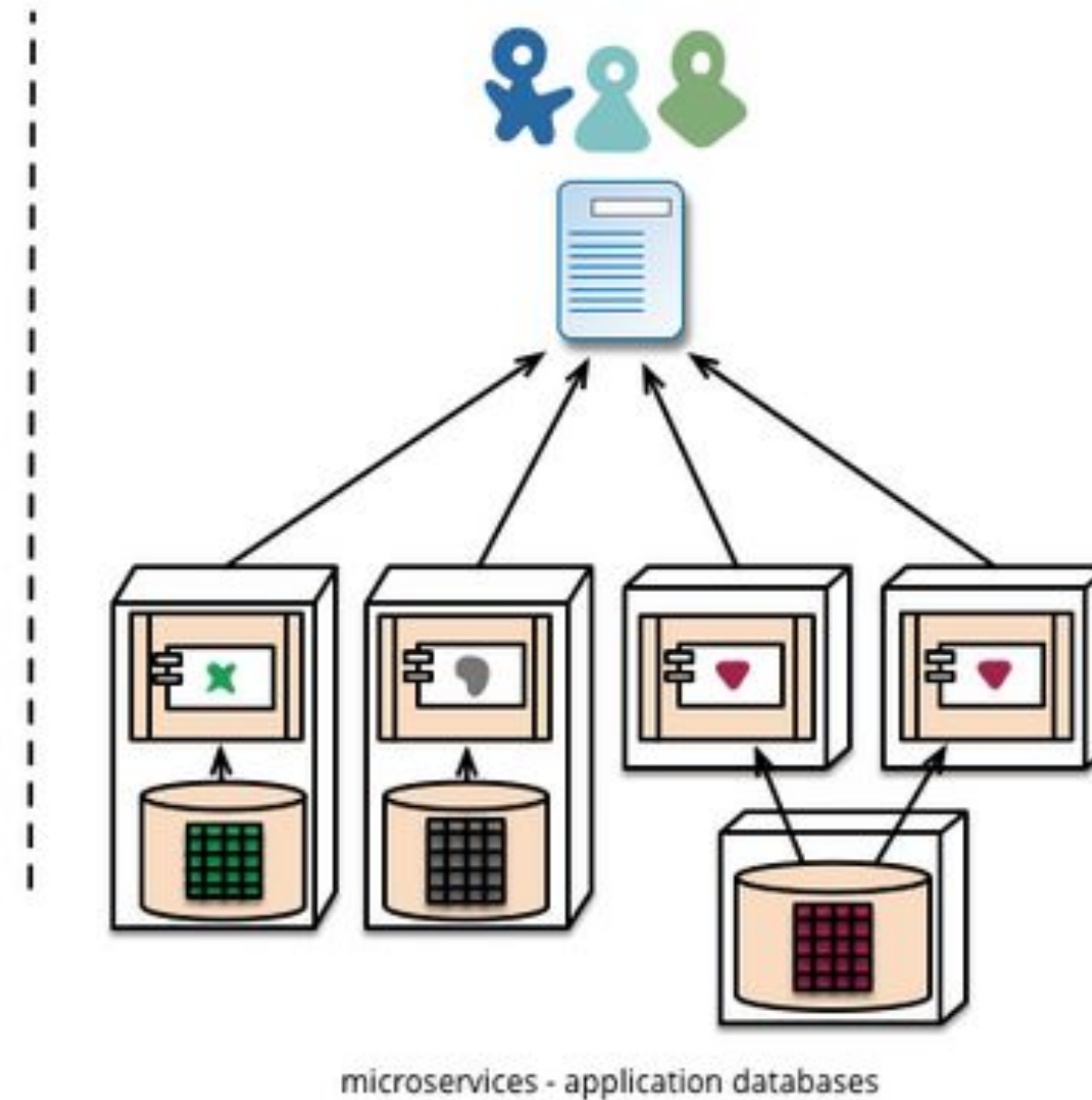
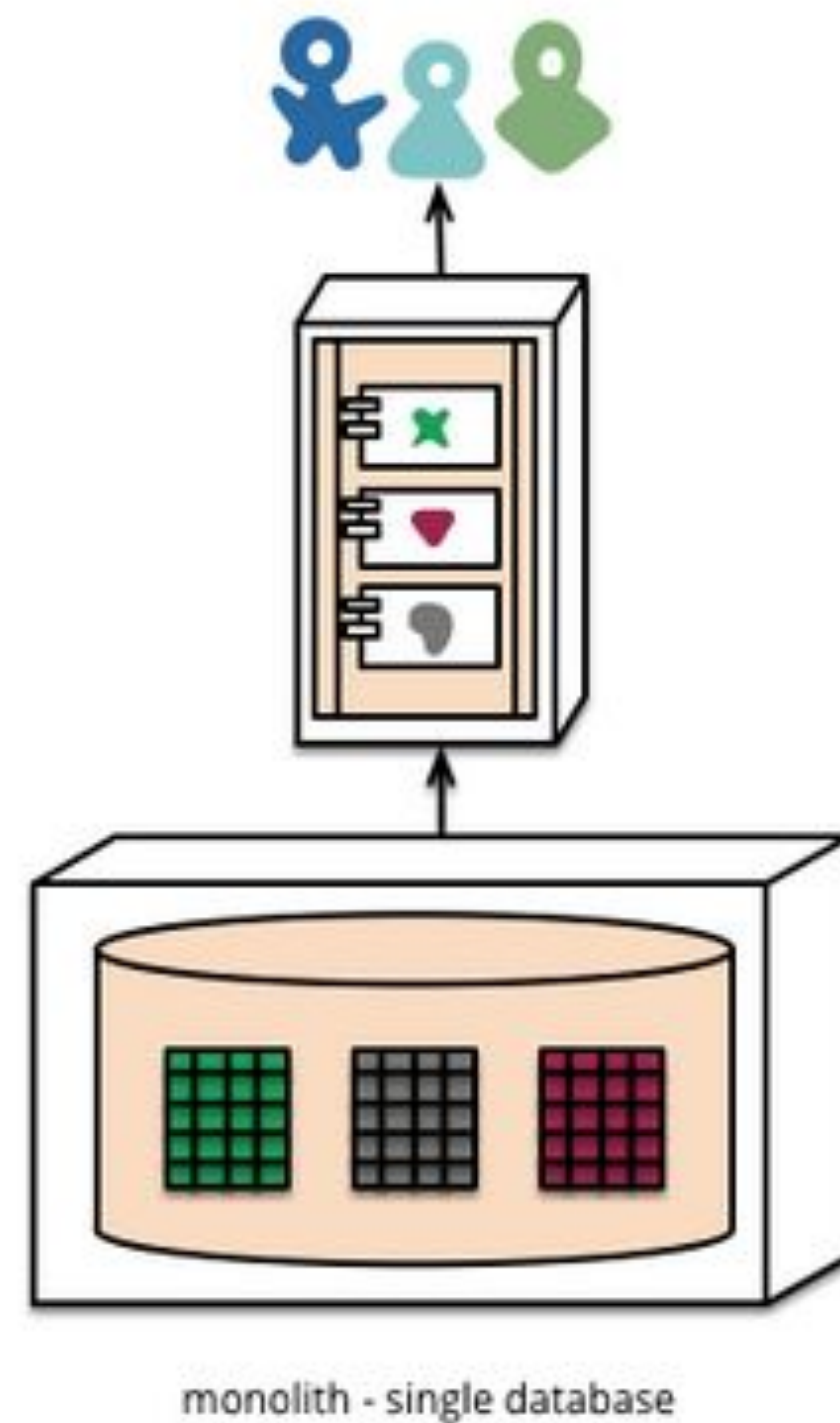
>> Arquitectura de Componentes



Un producto software puede componerse de múltiples aplicativos o elementos reutilizables.

Las relaciones que se establecen entre todos estos componentes conforman la Arquitectura de Componentes del producto

>> Arquitectura de Componentes

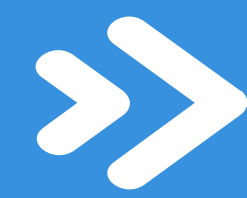


El término “arquitectura de **microservicios**” ha surgido en los últimos años para describir una forma particular de diseñar aplicaciones de software como conjuntos de servicios funcionalmente autónomos y desplegados independientemente en contraposición con la “arquitectura monolítica” basada en grandes productos software multifuncionales cuyos módulos están fuertemente acoplados.

¿Qué es la Arquitectura del Software?

El objetivo del diseño del software es concretar y aterrizar los componentes especificados en el análisis.

- **Arquitectura de Integración**
- **Arquitectura de Componentes**
- **Arquitectura de Desarrollo**



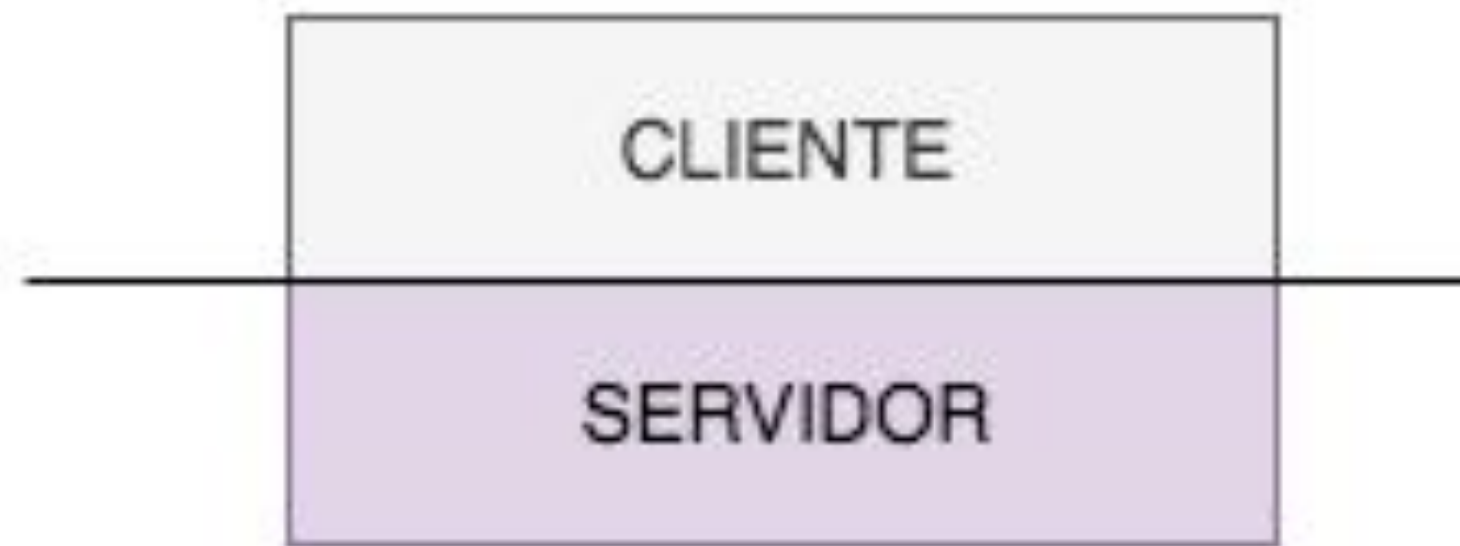
Niveles, Capas, Cebollas y Hexágonos

La arquitectura de desarrollo ofrece al desarrollador las “estanterías” en las que colocar las clases o componentes codificados.

Las “estanterías” son las librerías, proyectos o binarios en las que se divide el código. La “estantería” en que colocar cada clase o componente viene dado por sus responsabilidades y relaciones:

- ❑ Arquitectura multi-nivel
- ❑ Arquitectura multi-capa
- ❑ Arquitectura en cebolla
- ❑ Arquitectura hexagonal



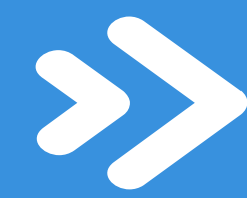


Una arquitectura multi-tier (o n-tier) hace referencia a una arquitectura en la que se expone la separación de sistema en varias capas físicas. Es decir, los distintos componentes están en máquinas separadas. En este tipo de arquitectura tenemos por ejemplo la de [cliente – servidor].

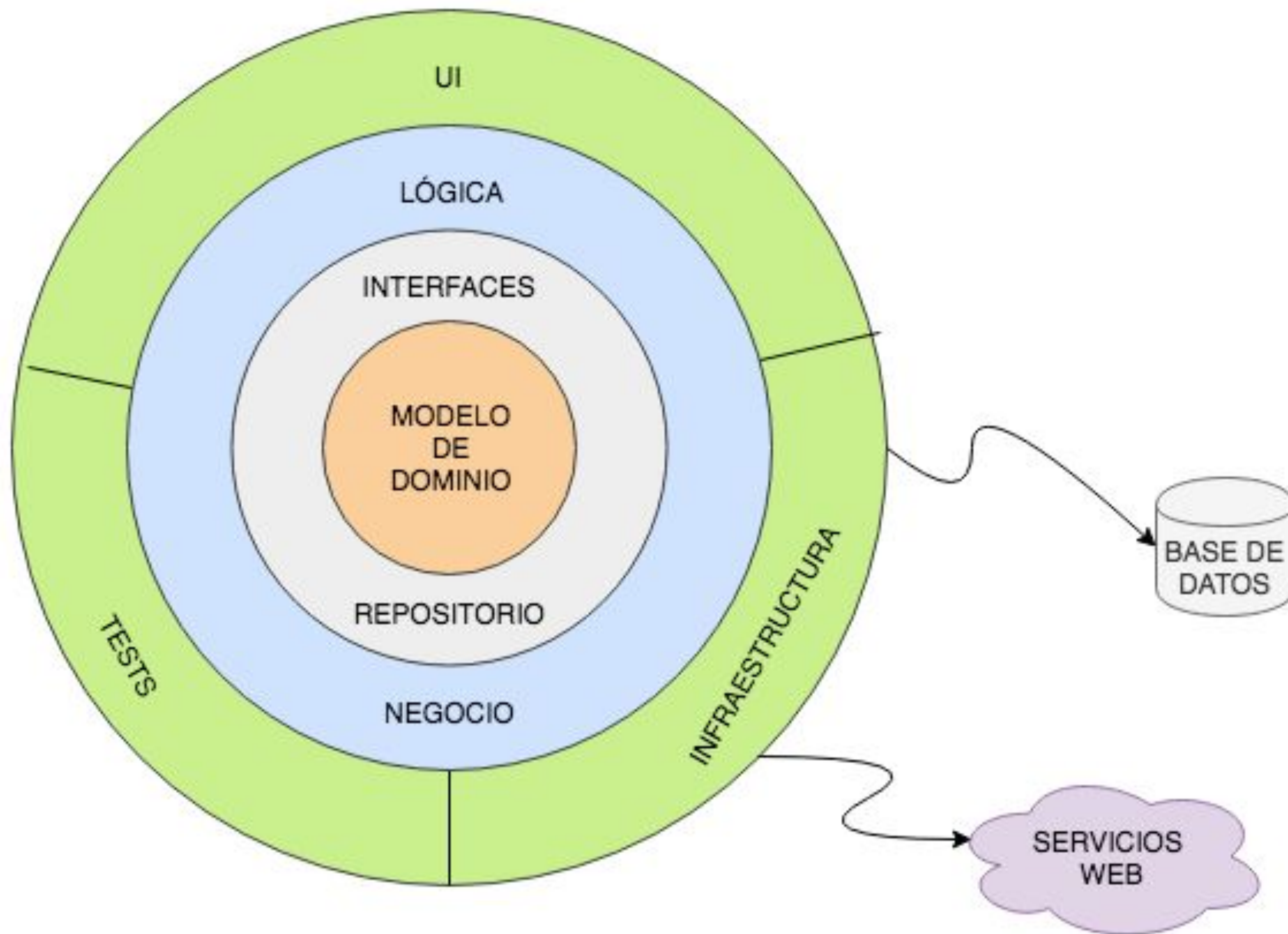


Una arquitectura multi-layer (o n-layer) refleja la separación lógica en capas de un sistema software. En este contexto una capa es simplemente un conjunto de clases, paquetes o subsistemas que tienen unas responsabilidades relacionadas dentro del funcionamiento del sistema. Estas capas están organizadas de forma jerárquica unas encima de otras y las dependencias siempre van hacia abajo. Es decir, que una capa concreta dependerá solamente de las capas inferiores, pero nunca de las superiores.

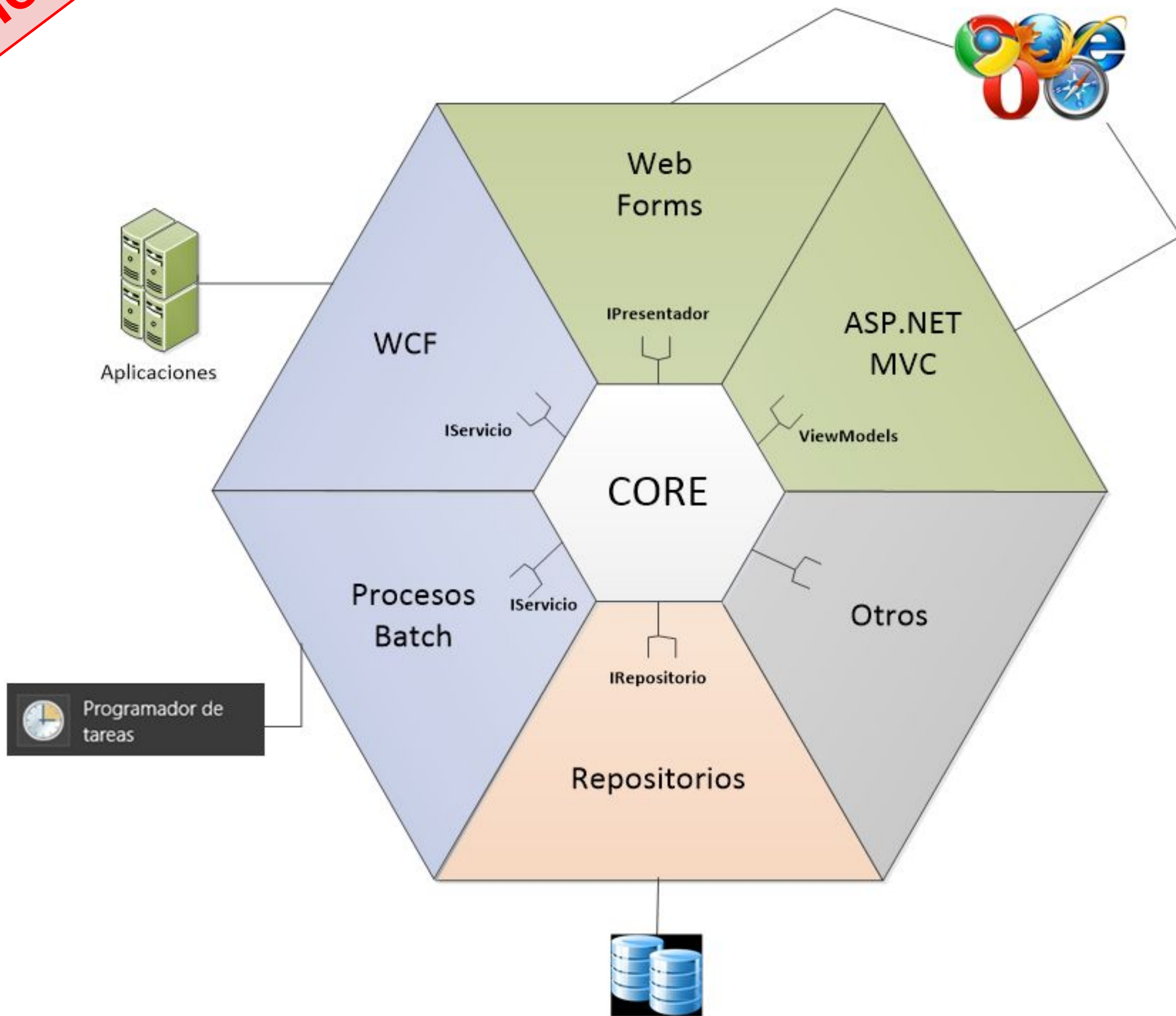
Cebolla



Arquitectura de Desarrollo



A grandes rasgos se trata de una arquitectura multicapa construida en torno a un modelo de dominio independiente de todo lo demás. Las dependencias van hacia el centro, por lo que todo depende de ese modelo de dominio. Alrededor del núcleo de modelo puede haber un número variable de capas, pero siempre debe cumplirse que las interfaces estén más cerca que las clases que las utilizan.



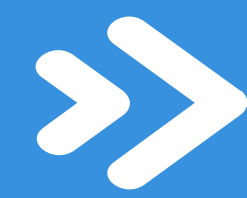
Ports and Adapters es más prescriptivo de la arquitectura en Cebolla definiendo y concretando puertos y adaptadores



Patrones de Presentación

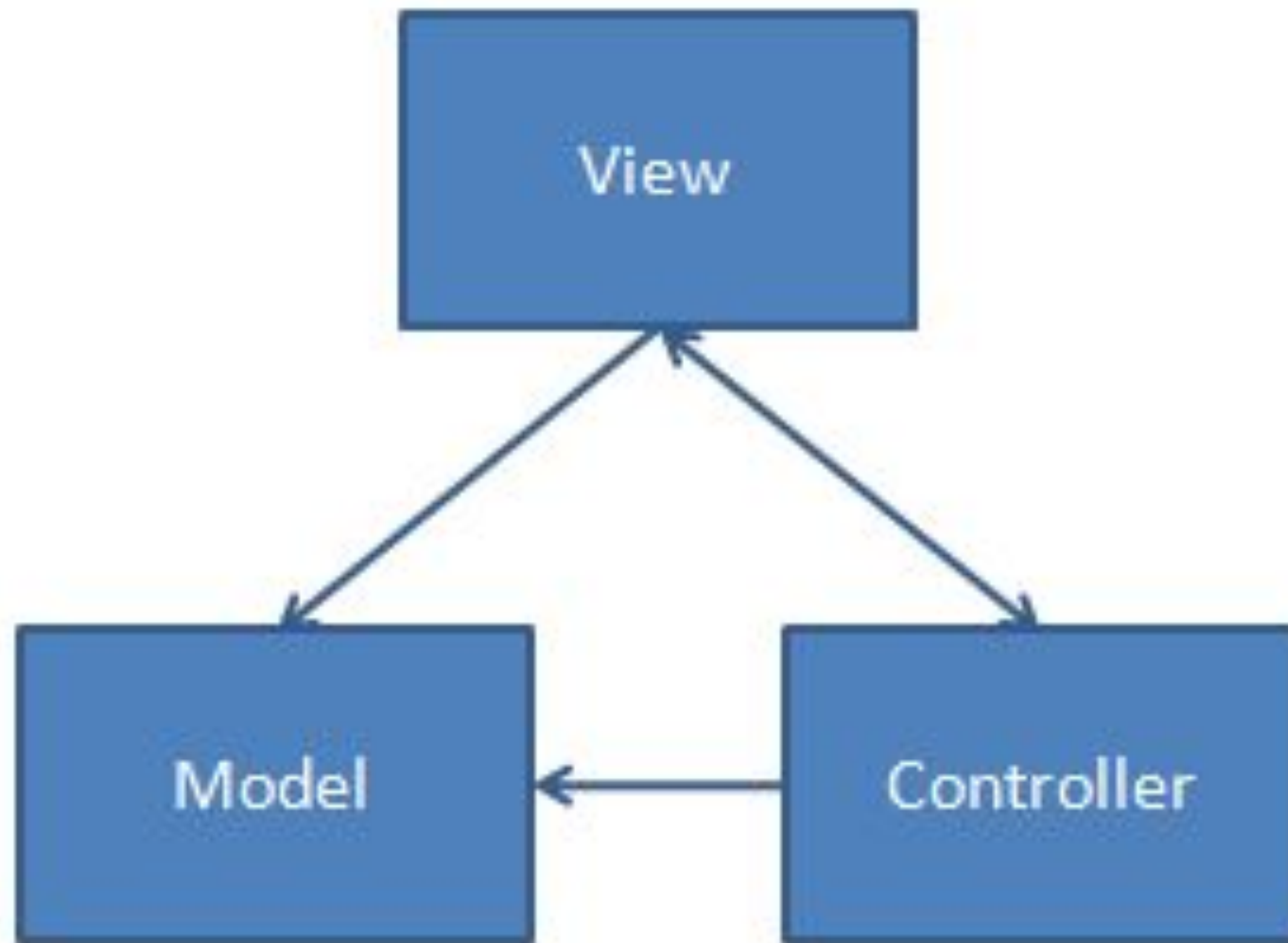
Existen diferentes patrones (soluciones recurrentes) para separar la lógica de presentación o vista (interfaz de usuario) con la lógica del negocio o modelo (entidades y servicios). Los patrones más habituales son el Modelo - Vista - Presentador (MVP) y el Modelo - Vista - Controlador (MVC):

- ❑ **Modelo.** Son el conjunto de clases que “modelan” el negocio o dominio. Por ejemplo: Enfermera, Farmaco, Paciente, Dosis, etc. Todas estas clases expresan lenguaje de negocio, en este caso hablan de la administración del medicamento.
- ❑ **Vista.** Son el conjunto de clases que interaccionan directamente con el usuario. Por ejemplo Aviso, Panel, Cabecera, Formulario. Estas clases no expresan lenguaje de negocio. Un aviso puede ser mostrado como una ventana emergente en un navegador o como una línea en rojo en una aplicación de consola.
- ❑ **Presentador / Controlador.** Son clases que actúan como intermediarios entre el Modelo y la Vista. En función de cómo implementan esta comunicación son definidos los Controladores y los Presentadores.

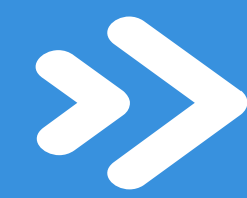


Arquitectura de Desarrollo

MVC



El controlador hace de enlace entre la vista y el modelo. La vista es la encargada de “renderizar” los objetos del modelo dotándolos de “inteligencia”. La lógica de la vista se encarga de transformar y adaptar los objetos del modelo del dominio para ser presentados al usuario.



Arquitectura de Desarrollo

Es una evolución de MVC, donde la vista no conoce el modelo.

El presentador hace de enlace entre la vista y el modelo, dotando de inteligencia a la vista. La vista se mantiene “tonta”.

La vista carece de lógica, su única labor es invocar los métodos del presentador y actualizarse con la respuesta.

El presentador escucha los eventos que se producen en la vista y ejecuta las acciones necesarias a través del modelo. Para ello, el presentador instanciará objetos del modelo e invocará sus métodos.

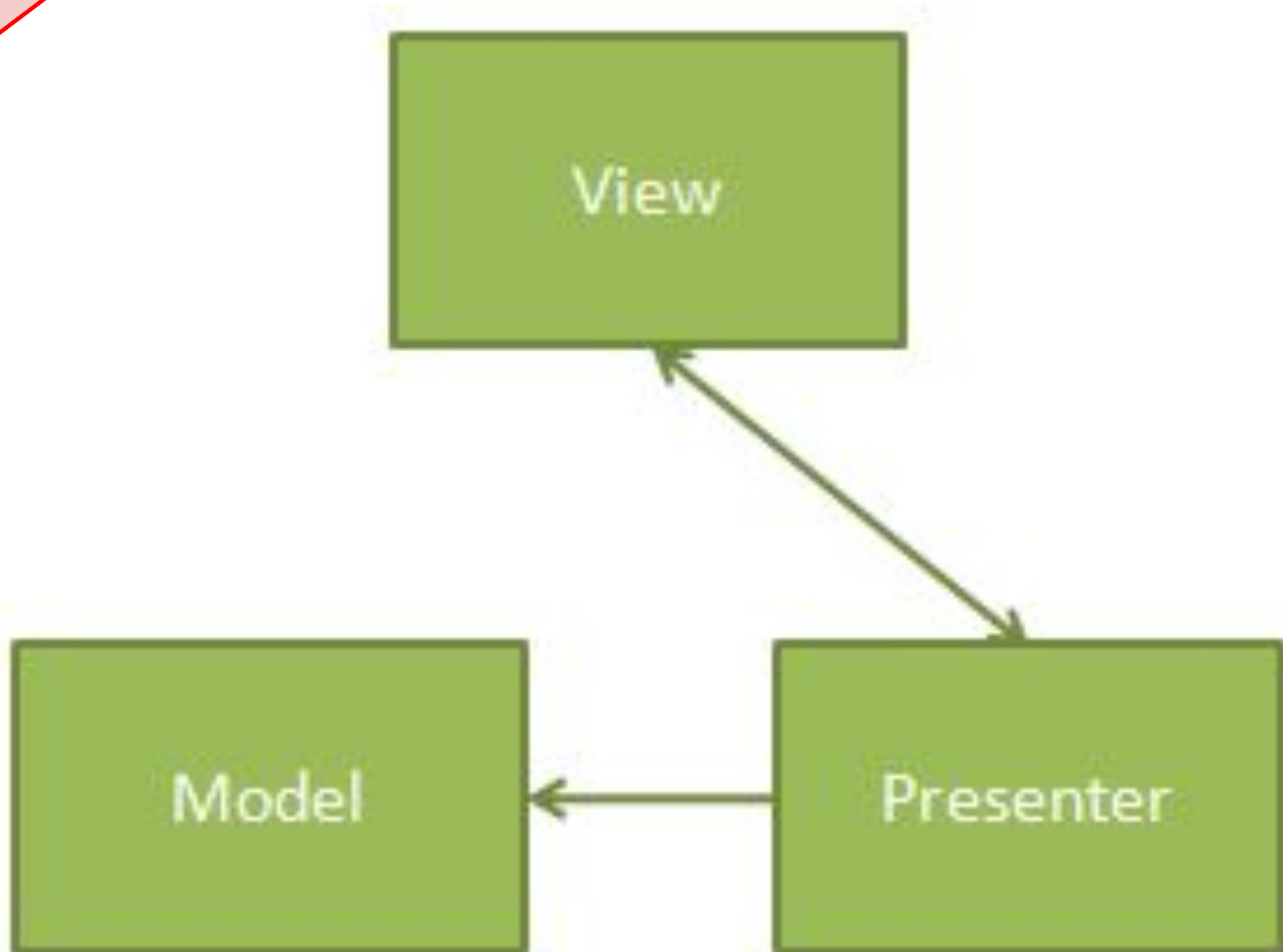




TABLA DE CONTENIDOS

01 **Arquitectura**

02 **Diseño**

03 **Interface y Experiencia de Usuario (UX/UI)**

Diseño

¿Qué es el Diseño del software?

El objetivo del diseño del software es imaginar, pensar o esbozar la implementación de los componentes y el software especificados en el análisis condicionado por las decisiones arquitectónicas tomadas.

¿Qué es el Diseño del software?

Arquitectura vs Diseño

Diferentes productos pueden hacer uso de una misma Arquitectura de Desarrollo, por ejemplo “2 niveles y 3 capas; de una misma Arquitectura de Presentación, por ejemplo patrón MVP ; y de una misma Arquitectura de Integración, por ejemplo servicios web como interfaces de comunicación. Todos estos productos comparten una misma “Arquitectura”.

Sin embargo, aunque estos productos compartan arquitectura, deberán necesariamente de dotarse de su propio diseño. Por ejemplo, el diseño de un producto software cuyo propósito sea gestionar planes de salud pública de cribado poblacional de cáncer se sustentará en clases como “Cancer”, “Citación”, “Carta”, “Kit”, etc.. El diseño de otro producto software cuyo propósito sea gestionar la farmacia hospitalario se sustentará en clases como “Prescripción”, “Fármaco”, “ReaccionAdversa”, etc. Ambos productos pueden compartir arquitectura, pero el diseño de los mismos es diferente.

¿Qué es el Diseño del software?

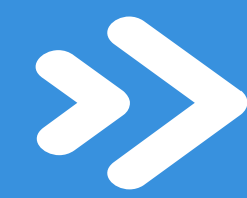
Existen técnicas y patrones que ayudan a diseñar la solución:

- Los **Principios de Diseño** son patrones habituales, soluciones recurrentes a problemas similares. Los Principios de Diseño son **útiles para imaginar** en la mente del diseñador las clases, estructuras y componentes necesarios. En el paradigma Orientado a Objetos existen Principios de Diseño que ayudan al programador o diseñador a reflexionar sobre la estructura de las clases y sus relaciones. Seguir los principios de diseño permiten generar código mantenible, extensible y testable, no seguirlos genera código difícil de entender y modificar. Estos principios guían a los diseñadores a “descubrir” las clases y componentes necesarios.
- Los **Diagramas UML** permiten materializar en una representación gráfica los diseños que habitan en la mente del diseñador. Son un **vehículo de comunicación** entre los miembros del equipo de desarrollo. A la hora de plasmar en un documento el diseño de las clases es habitual recurrir a la utilización de diagramas UML.
- El **Desarrollo Guiado por Pruebas** obliga a **poner a prueba los diseños** aterrizándolos directamente en código. Las pruebas ofrecen un rápido feedback al diseñador-codificador permitiéndole mejorar los diseños iniciales. Las pruebas ejercitan el código, permitiendo al programador refinar y mejorar sus diseños iniciales.

¿Qué es el Diseño del software?

El objetivo del diseño del software es concretar y aterrizar cómo se implementarán los componentes y el software especificados en el análisis condicionado por las decisiones arquitectónicas tomadas.

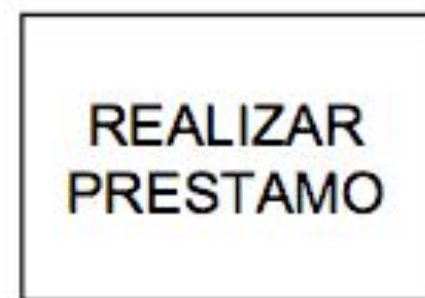
- **Diseño Estructurado**
- **Principios de Diseño**
- **Utilización de diagramas UML**
- **Desarrollo guiado por pruebas**



Diseño Estructurado

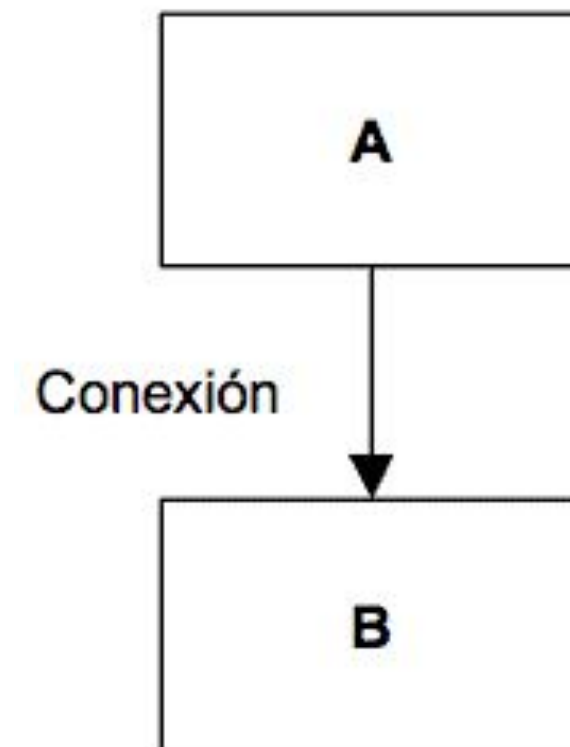
Esta técnica de diseño es el complemento del llamado Análisis Estructurado. Ambas técnicas coinciden en el empleo de los Diagramas de Flujos de Datos (DFD) como medio fundamental de representación del modelo funcional del sistema. La tarea de diseño consiste en pasar de los DFS a los diagramas de estructuras. El Análisis Estructurado y el Diseño Estructurado, muy utilizados en los 70s, suelen ir asociados al paradigma imperativo de codificación.

El objetivo de este diagrama es representar la estructura modular del sistema o de un componente del mismo y definir los parámetros de entrada y salida de cada uno de los módulos



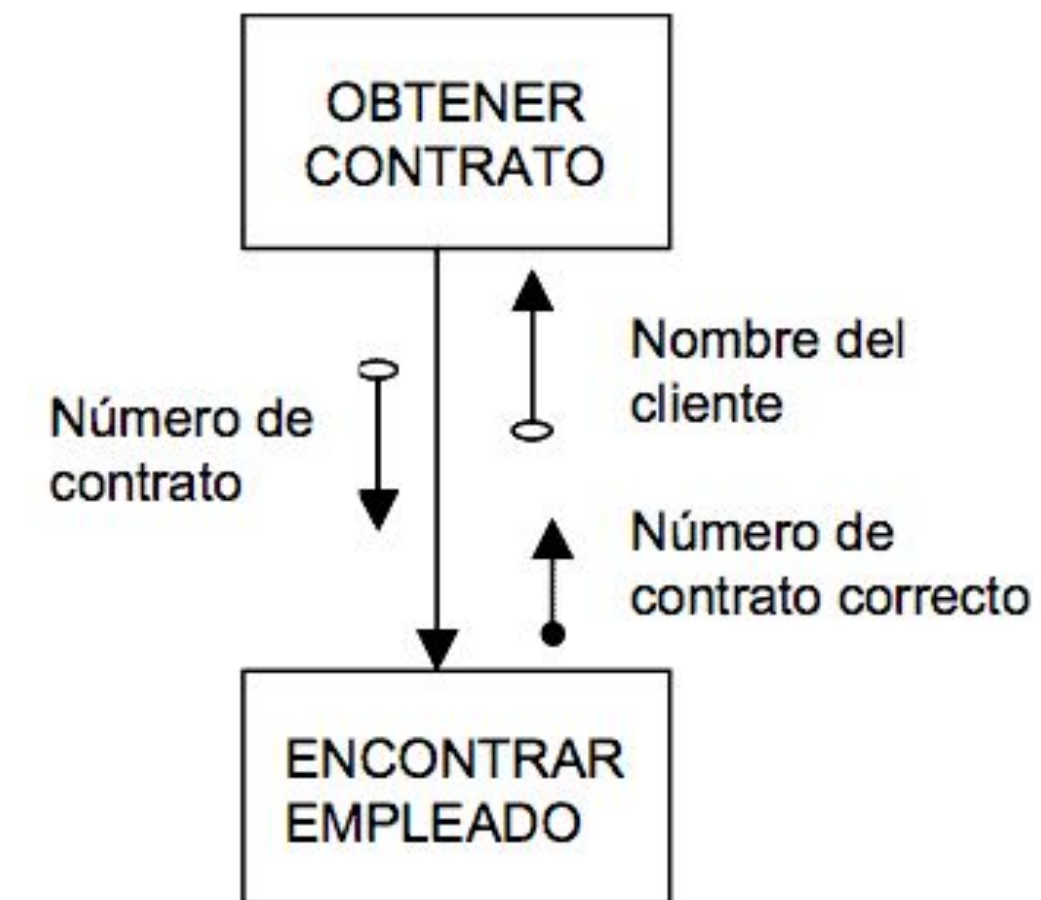
MÓDULO

División del software clara y manejable con interfaces perfectamente definidas. Un módulo puede representar un programa, subprograma o rutina dependiendo del lenguaje a utilizar.



CONEXIÓN

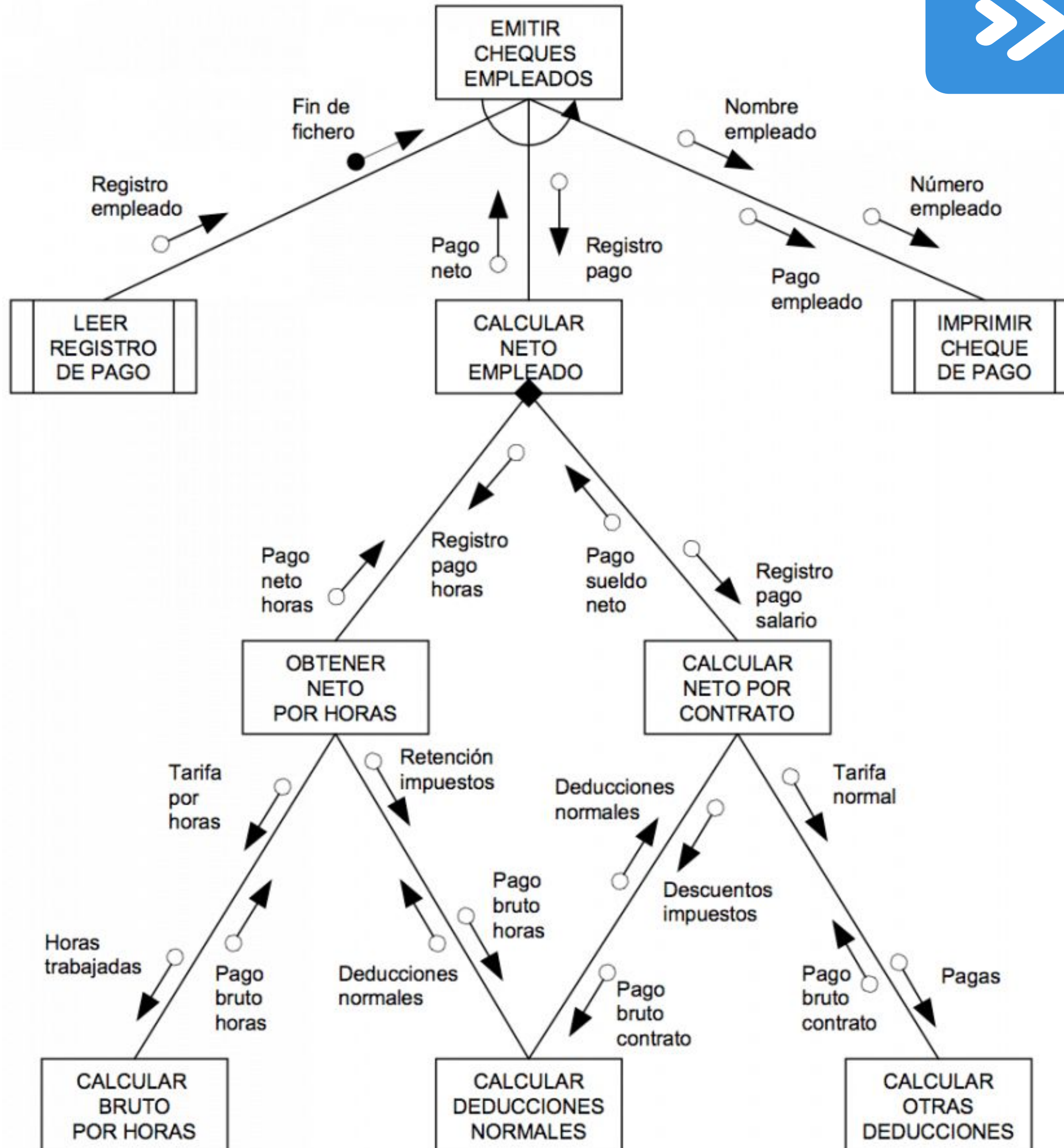
Representa una llamada de un módulo a otro



PARÁMETROS

Información que se intercambia entre los módulos. Pueden ser de dos tipos en función de la clase de información a procesar

» Diseño Estructurado



¿Qué es el Diseño del software?

El objetivo del diseño del software es concretar y aterrizar cómo se implementarán los componentes y el software especificados en el análisis condicionado por las decisiones arquitectónicas tomadas.

- **Diseño Estructurado**
- **Principios de Diseño**
- **Utilización de diagramas UML**
- **Desarrollo guiado por pruebas**



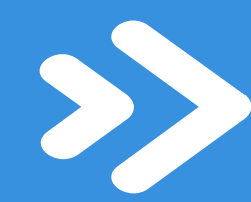
Existen **Principios de Diseño** que ayudan al programador/diseñador a reflexionar sobre la estructura de las clases y sus relaciones.

Seguir los principios de diseño permiten generar código mantenible, extensible y testable, no seguirlos genera código difícil de entender y modificar.



En 1975, Larry Constantine y Edward Yourdon publicaron “Structured Design”. En él mencionan la importancia de la alta cohesión y del bajo acoplamiento a la hora de diseñar código mantenible (fácil de corregir) y extensible (fácil de ampliar). Además, si el código ofrece una alta cohesión y un bajo acoplamiento, entonces el código es fácilmente testable.

- **Alta Cohesión.** Si dos funciones deben trabajar en sintonía para alcanzar un objetivo, entonces un buen diseño debería encapsular estas funciones en la misma clase. Por el contrario, si dos segmentos de código realizan trabajos diferentes, deberían encapsularse en distintas funciones.
- **Bajo Acoplamiento.** Si una función del módulo A invoca a una función del módulo B, entonces un cambio en el código de la función de B podría afectar a la función de A. Decimos entonces que el módulo A está acoplado al módulo B.



Principios de Diseño

A la rueda trasera
le falta cohesión y
le sobra
acoplamiento





-  **Las Cuatro Reglas del Diseño Simple**
-  **Principios SOLID**
-  **Patrones de Diseño**
-  **Inversión del Control**
-  **Diseño guiado por el Dominio**
-  **Otros principios**

- >> **Las Cuatro Reglas del Diseño Simple**
- >> Principios SOLID
- >> Patrones de Diseño
- >> Inversión del Control
- >> Diseño guiado por el Dominio
- >> Otros principios

>> Las Cuatro Reglas del Diseño Simple

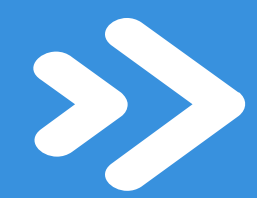
Kent Beck propuso las Cuatro Reglas del Diseño Simple mientras desarrollaba la metodología XP a finales de los 90s:

1. El código debe “**pasar**” **los tests**. En primer lugar, por supuesto, la aplicación debe hacer lo que se espera que haga. Para verificar que el código funciona es imprescindible cubrirlo con tests automatizados.
2. El código debe **revelar su intención**. A continuación, el código debe ser fácil de entender. Hacer uso de un buen naming es fundamental.
3. El código debe **evitar duplicaciones**. Cada pieza de código debe tener una única, no ambigua y representativa identidad dentro del sistema.
4. El código debe tener el **menor número de elementos** posible. Por último, todo lo que no cumpla con las tres reglas anteriores debe eliminarse. Esta regla pretende simplificar los diseños, eliminando todas las abstracciones innecesarias. Si con únicamente dos clases, el código pasa las pruebas, revela su intención y evita las repeticiones, entonces esta solución es preferible a otra que haga uso de tres clases para obtener lo mismo.

- >> Las Cuatro Reglas del Diseño Simple
- >> Principios SOLID
- >> Patrones de Diseño
- >> Inversión del Control
- >> Diseño guiado por el Dominio
- >> Otros principios

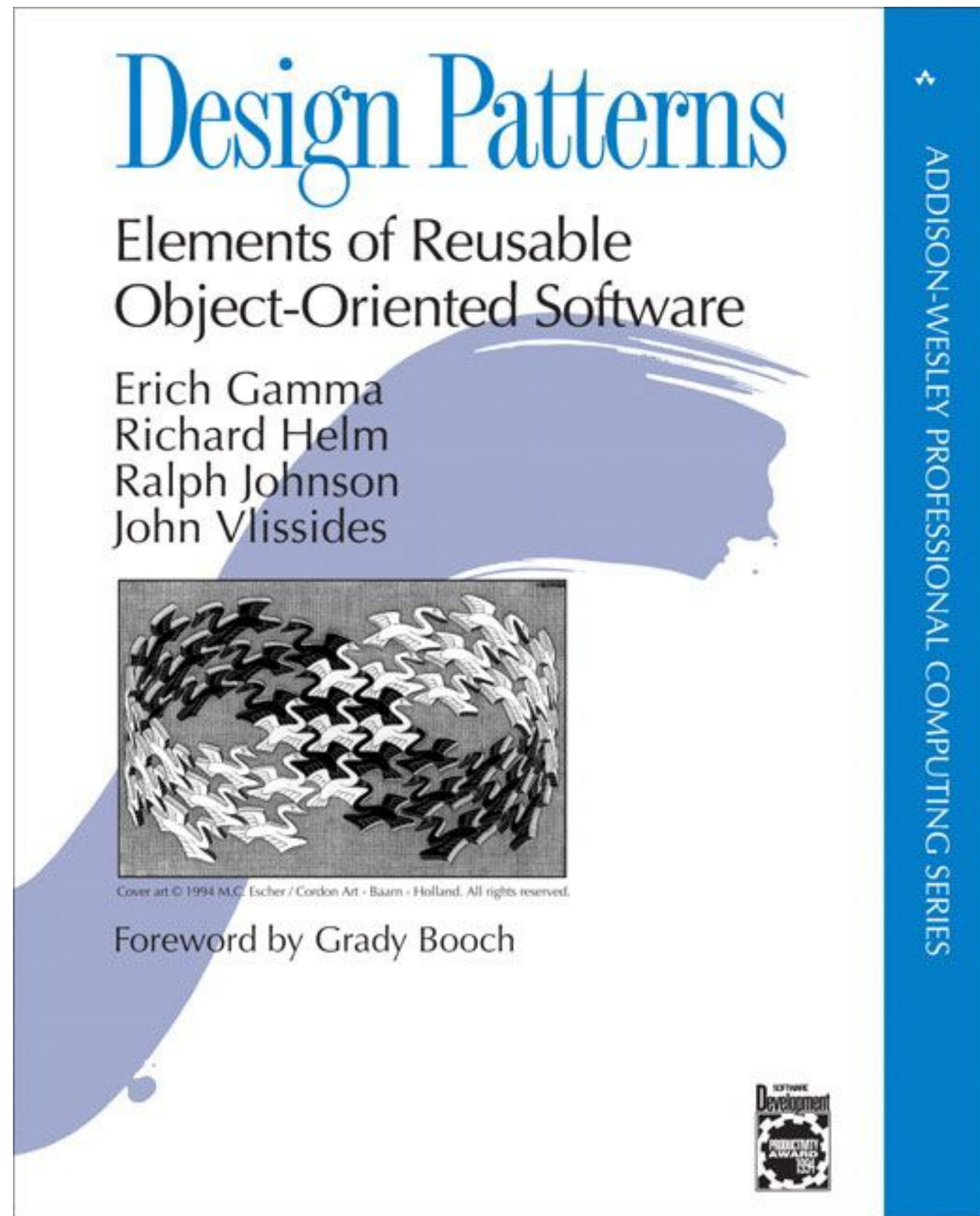
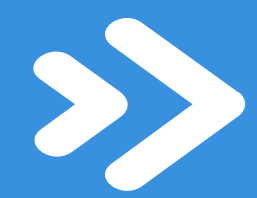


SOLID es un acrónimo que acuñó Michael Feathers, basándose en los principios de la programación orientada a objetos que Robert C. Martin había recopilado en el año 2000 en su artículo “[Design Principles and Design Patterns](#)”.



- ❑ **Single responsibility principle** o Principio de responsabilidad única. Solo debe existir un motivo para modificar una clase, o dicho de otra forma, una clase solo debe ser responsable de una funcionalidad.
- ❑ **Open/closed principle** o Principio de abierto (a la extensión) / cerrado (a la modificación). Establece que las entidades (clases y módulos) deben estar abiertos para su extensión, pero cerrados para su modificación. En lugar de modificar el método de una clase añadiendo una nueva condición puede resultar preferible crear una nueva clase que extienda la anterior sobrescribiendo el método con el nuevo comportamiento. Para cambiar el comportamiento del software es preferible escribir nuevo código a modificar el ya existente.
- ❑ **Liskov substitution principle** o Principio de sustitución de Liskov (*Barbara Liskov*). Las clases “hijas” o derivadas no pueden conocer a las clases “padres” o base.
- ❑ **Interface segregation principle** o Principio de segregación del interface. Los clientes no deberían verse forzados a depender de interfaces que no usan. En lugar de añadir un nuevo método a una clase o interfaz, puede resultar más adecuado crear una nueva clase o interface.
- ❑ **Dependency inversion principle** o Principio de inversión de dependencia. Las clases deben depender de abstracciones no de implementaciones. Se debe programar contra interfaces, no contra implementaciones. Esto permite sustituir componentes en el diseño de forma sencilla.

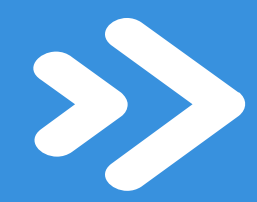
- >> Las Cuatro Reglas del Diseño Simple
- >> Principios SOLID
- >> **Patrones de Diseño**
- >> Inversión del Control
- >> Diseño guiado por el Dominio
- >> Otros principios



Los patrones de diseño ofrecen soluciones a problemas recurrentes en el desarrollo del software. Normalmente constan de una serie de pautas a seguir (una receta) que resuelven un problema concreto que ha sido ya probado y documentado por gran parte de la comunidad.

Los principales patrones de diseño en el software fueron recopilados en 1994 en el libro “Design Patterns” escrito por el grupo Gang of Four (GoF) compuesto por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides. En él se recogen los 23 patrones de diseño más comunes en el desarrollo de software.

- >> Las Cuatro Reglas del Diseño Simple
- >> Principios SOLID
- >> Patrones de Diseño
- >> **Inversión del Control**
- >> Diseño guiado por el Dominio
- >> Otros principios



Inversión del Control

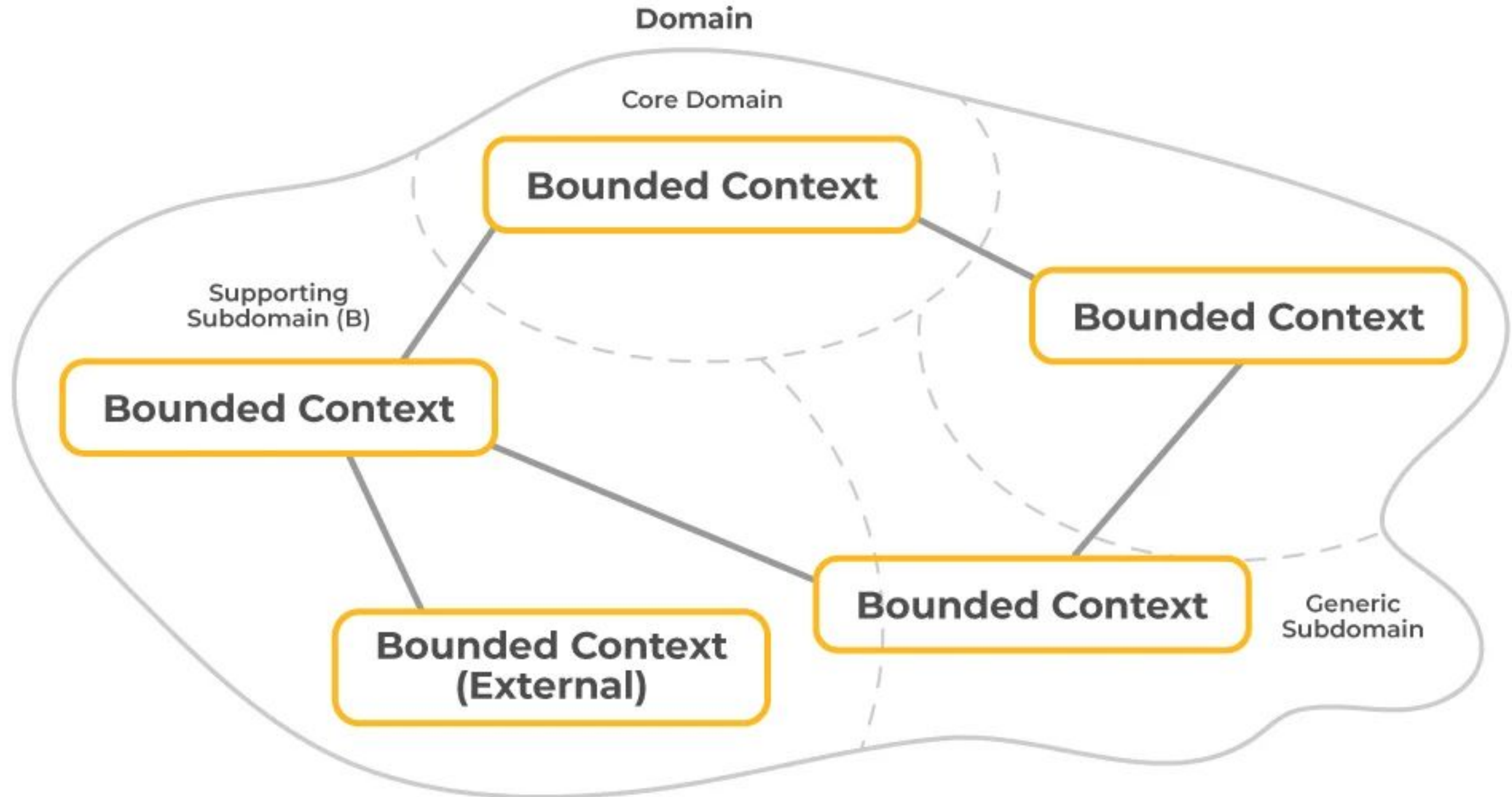


- >> Las Cuatro Reglas del Diseño Simple
- >> Principios SOLID
- >> Patrones de Diseño
- >> Inversión del Control
- >> **Diseño guiado por el Dominio**
- >> Otros principios



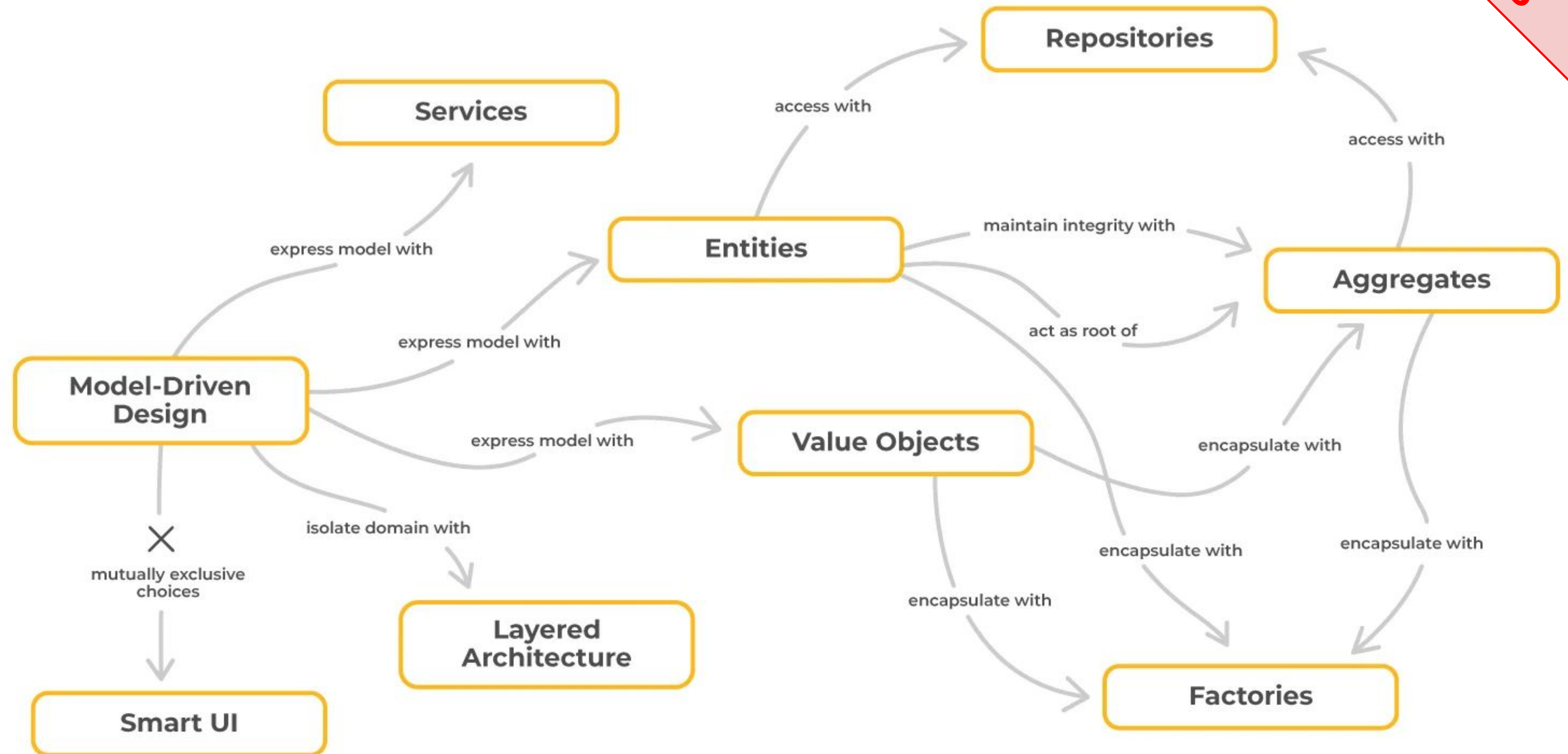
Diseño guiado por Dominio (DDD)

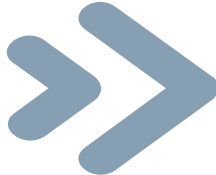
Patrones
Estratégicos

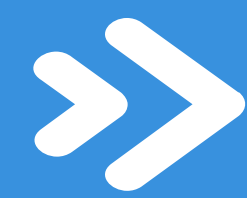


>> Diseño guiado por Dominio (DDD)

Patrones
Tácticos



-  **Las Cuatro Reglas del Diseño Simple**
-  **Principios SOLID**
-  **Patrones de Diseño**
-  **Inversión del Control**
-  **Diseño guiado por el Dominio**
-  **Otros principios**



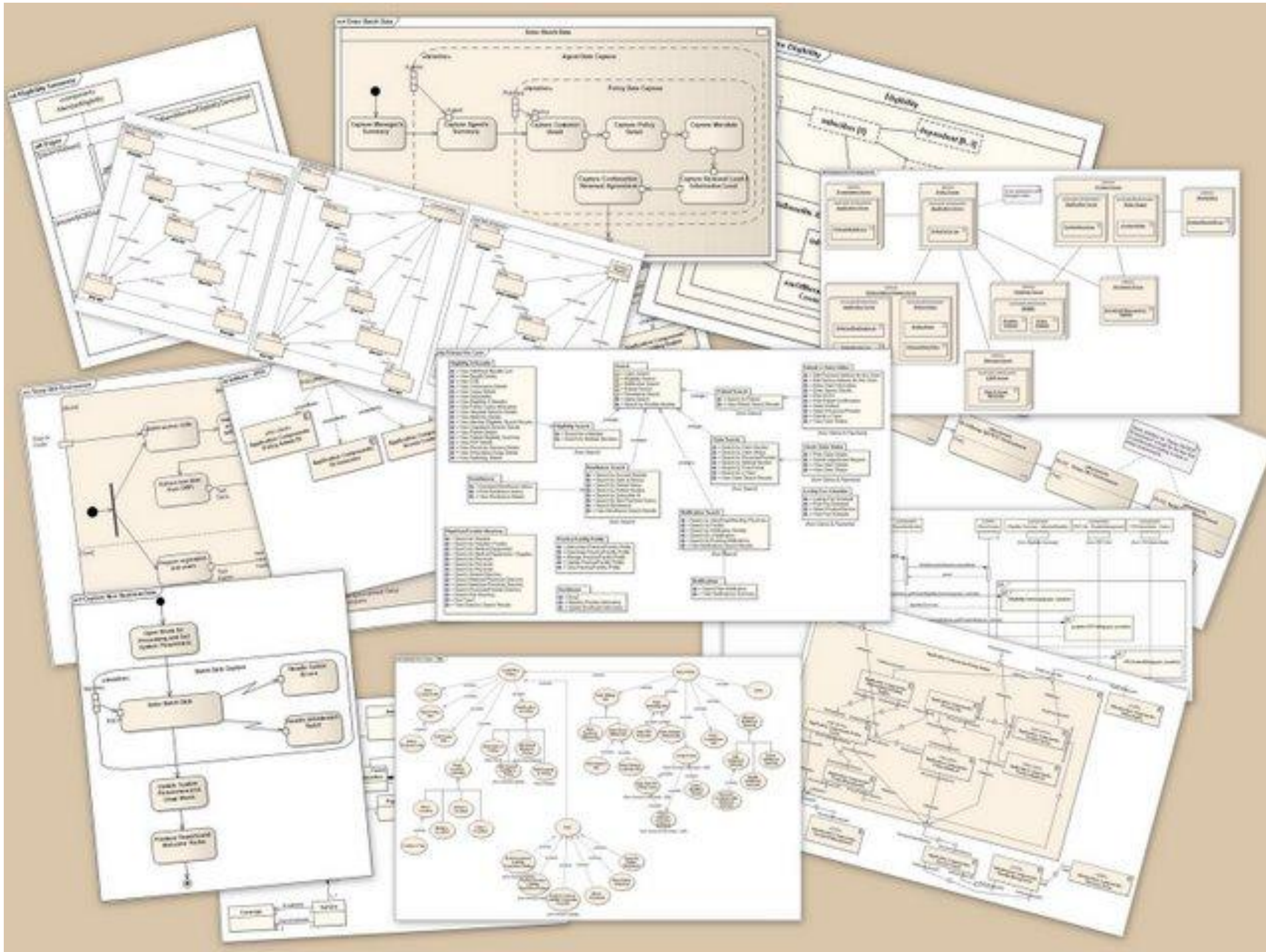
- ❑ **KISS.** “Keep It Simple, Stupid”. El principio KISS establece que la mayoría de sistemas funcionan mejor si se mantienen simples que si se hacen complejos; por ello, la simplicidad debe ser mantenida como un objetivo clave del diseño, y cualquier complejidad innecesaria debe ser evitada.
- ❑ **DRY.** “Don’t Repeat Yourself”. Promueve la reducción de la duplicación. Cada pieza de funcionalidad debe tener una única, no ambigua y representativa identidad dentro del sistema. Para facilitar el mantenimiento del código debe evitarse duplicarlo
- ❑ **LoD.** “Ley de Demeter” o “Principio del Menor Conocimiento”, dado un objeto este debería asumir tan poco como sea posible sobre la estructura o propiedades de cualquier otro (incluyendo sus subcomponentes)
- ❑ **DAMP.** DAMP significa "Descriptive and Meaningful Phrases" promueve el uso de nombres largos y descriptivos de variables y funciones.

¿Qué es el Diseño del software?

El objetivo del diseño del software es concretar y aterrizar cómo se implementarán los componentes y el software especificados en el análisis condicionado por las decisiones arquitectónicas tomadas.

- **Diseño Estructurado**
- **Principios de Diseño**
- **Utilización de diagramas UML**
- **Desarrollo guiado por pruebas**

» Diagramas UML



UML, Lenguaje Unificado de Modelado, es un lenguaje gráfico para visualizar, especificar, construir y documentar un sistema.

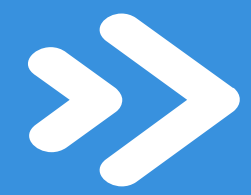
El lenguaje de modelado incluye un conjunto de notaciones gráficas que permiten crear modelos visuales de sistemas. UML admite hasta 14 tipos de diagramas.



Uso de los Diagramas UML

Existen tres formas diferentes de hacer uso de diagramas UML durante el diseño del software:

- ❑ **Lenguaje** de programación. Si el diagrama es muy riguroso y detallado, herramientas informáticas pueden transformar estos diagramas en código ejecutable. A pesar de los intentos de la industria en el desarrollo de lenguajes gráficos de programación, estos no terminan de implantarse de forma generalizada.
- ❑ **Planos** del código. Disponer de planos del software permitiría a la industria la especialización de roles en diferentes equipos de personas, el grupo de diseñadores entrega los planos del software que el grupo de codificadores se encarga de transformar en código. Para ello es necesario el desarrollo de diagramas suficientemente rigurosos y detallados. Los planos tienen como objetivo transmitir información detallada sobre el código, ya sea en documentos en papel o como un navegador gráfico interactivo. Los planos pueden mostrar todos los detalles de una clase en una forma gráfica que sea más fácil de entender para los desarrolladores.
- ❑ **Boceto** del código. Los desarrolladores usan UML para ayudar a comunicar algunos aspectos de un sistema. En frente del boceto el equipo de desarrollo resuelve algunos problemas del código que están a punto de escribir. El boceto ayuda a dirigir el debate y ayuda a comunicar los diferentes puntos de vista. Estos diagramas no son rigurosos y suelen evitar “enfangarse” en los detalles de la implementación.

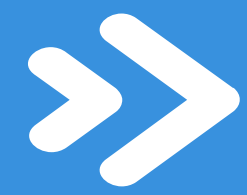


Uso de los Diagramas UML

*“[...] Primero, tenga en cuenta para qué está dibujando los diagramas. El valor principal es la comunicación. La comunicación eficaz significa seleccionar cosas importantes y descuidar las menos importantes. [...] El problema surge cuando las personas piensan que el diseño está terminado y luego no toman el conocimiento que adquirieron a través de la codificación y lo ejecutan nuevamente en el diseño.
[...] Cambiar el diseño no significa necesariamente cambiar los diagramas. Es perfectamente razonable dibujar diagramas que le ayuden a comprender el diseño y luego desechar los diagramas. Recuerde que el código es el depósito de información detallada, los diagramas actúan para resumir y resaltar cuestiones importantes.
[...]”*

Martin Fowler, autor de UML Distilled (1997)

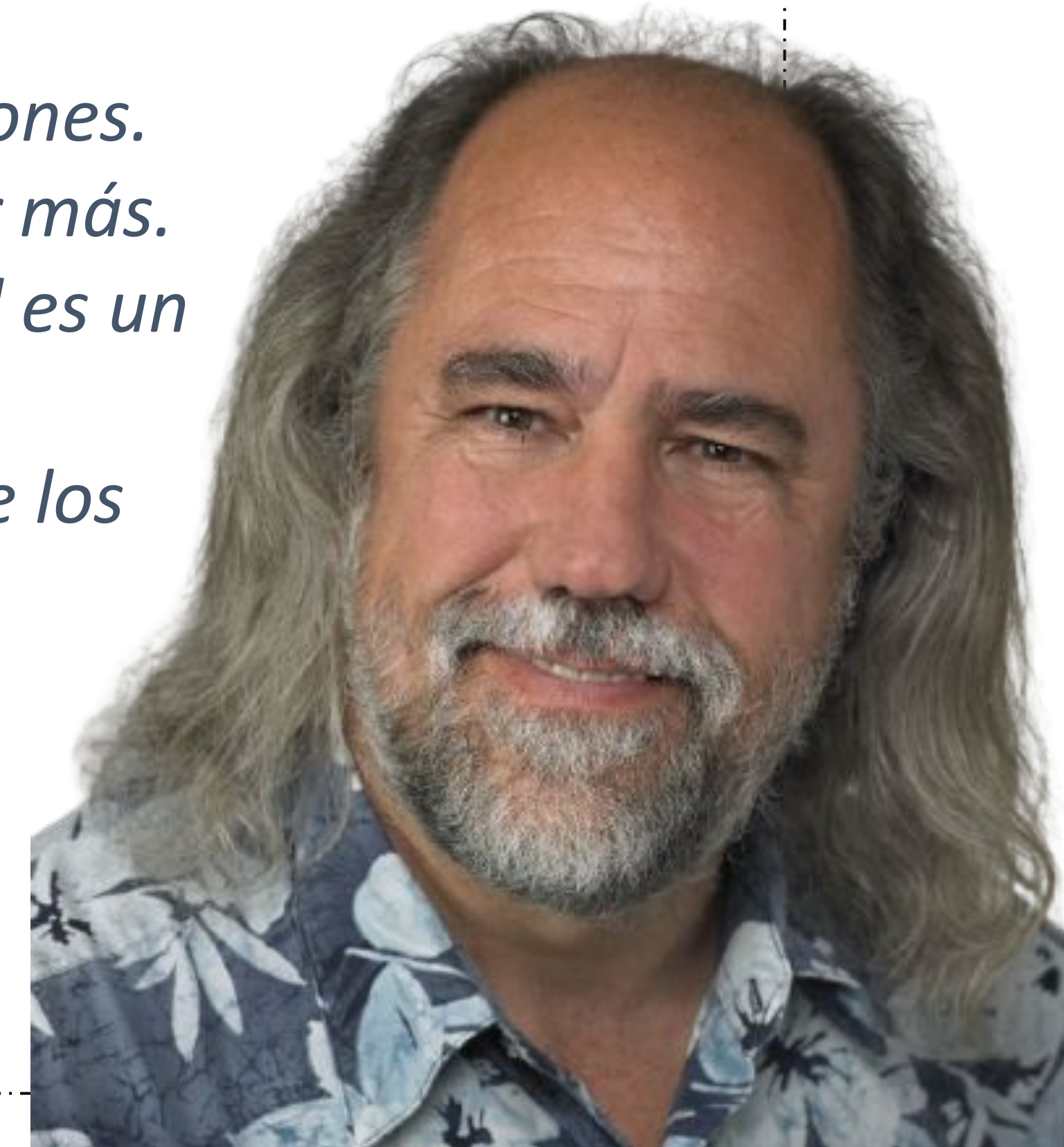


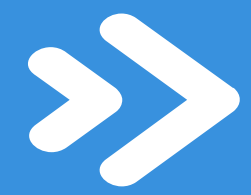


Uso de los Diagramas UML

- *UML debería usarse para razonar y discutir sobre alternativas. Crea algunos diagramas del sistema. Comprueba como de bien responden a los casos de uso que deberían cumplir. Después tira los diagramas y escribe el código que corresponde al mejor diseño*
- *Cuando empezamos con el UML, nunca tuvimos la intención de usarlo como un lenguaje de programación*
- *No conseguimos solucionar bien la parte de modelar colaboraciones. Los diagramas de componentes y despliegue necesitan madurar más. El metamodelo de UML ha crecido demasiado, esta complejidad es un error innecesario.*
- *Deberías usar una notación gráfica para aquellos aspectos sobre los que no se puede discutir fácilmente mirando el código*

Grady Booch, “padre” del UML



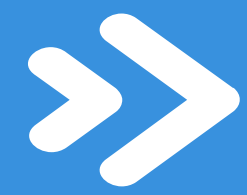


Uso de los Diagramas UML

- *UML no es la solución mágica (la “silver bullet”) que nos vendieron hace más de diez años. Tampoco es tan malo como algunos académicos, agilistas y competidores intentaron convencernos hace cinco.*
- *Hay dos grandes problemas a arreglar: la complejidad de la especificación UML misma y un mecanismo que permita partir UML en subconjuntos coherentes que puedan usarse independientemente para solucionar las necesidades de modelado de dominios concretos.*
- *UML se ha convertido en algo complejo y torpe de usar. El 80% del software sólo necesita el 20% del lenguaje UML*

Ivar Jacobson, “padre” del UML





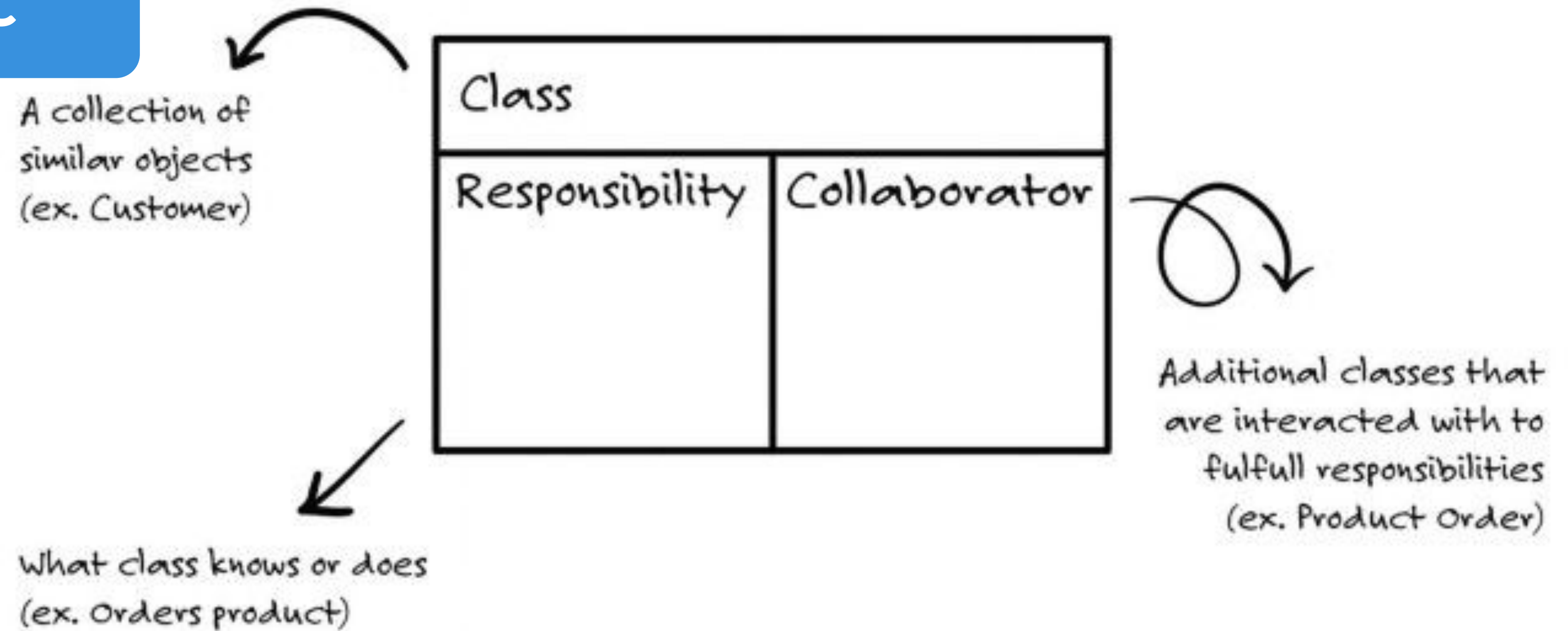
Uso de los Diagramas UML

- *UML es, de hecho, tan complejo como un lenguaje de programación grande y críptico, con un uso generoso de “\$” y “#” y “–” y “triángulos sólidos sin cola” y rectángulos y diamantes y líneas sólidas y líneas punteadas y elipses sólidas y elipses punteadas y flechas de todo tipo y palabras reservadas como “const” y “sorted” (no confundir con “ordered”) y semánticas distintas para una clase dependiendo si su nombre aparece en letra “romana” o “itálica”. ¡Pero al menos un lenguaje de programación, incluso el peor de ellos, es ejecutable! Aquí hay que aprender toda esa complejidad monstruosa para construir diagramas de un posible sistema futuro.*

Bertrand Meyer, creador del lenguaje Eiffel



>> Tarjetas CRC



Las tarjetas Clase-Responsabilidad-Colaborador fueron propuestas por Kent Beck y Brad Cunningham. Las tarjetas CRC ayudan a los programadores menos experimentados a conceptualizar los objetos de la solución. Las tarjetas CRC ayudan a “**bocetar**” el diseño de forma mucho menos prescriptiva y detallada que los diagramas UML de clases.

Centraremos nuestra atención en los dos más representativos y utilizados:

- ❑ **Diagrama de clases.** Es un diagrama de estructura que muestra los métodos, propiedades y relaciones de una clase.
- ❑ **Diagrama de secuencia.** El diagrama de secuencia es un tipo de diagrama de interacción cuyo objetivo es describir el comportamiento dinámico del sistema de información haciendo énfasis en la secuencia de los mensajes intercambiados por los objetos.

>> Diagrama de Clases

Las “Cajas” representan **clases** y las “flechas” sus **relaciones**

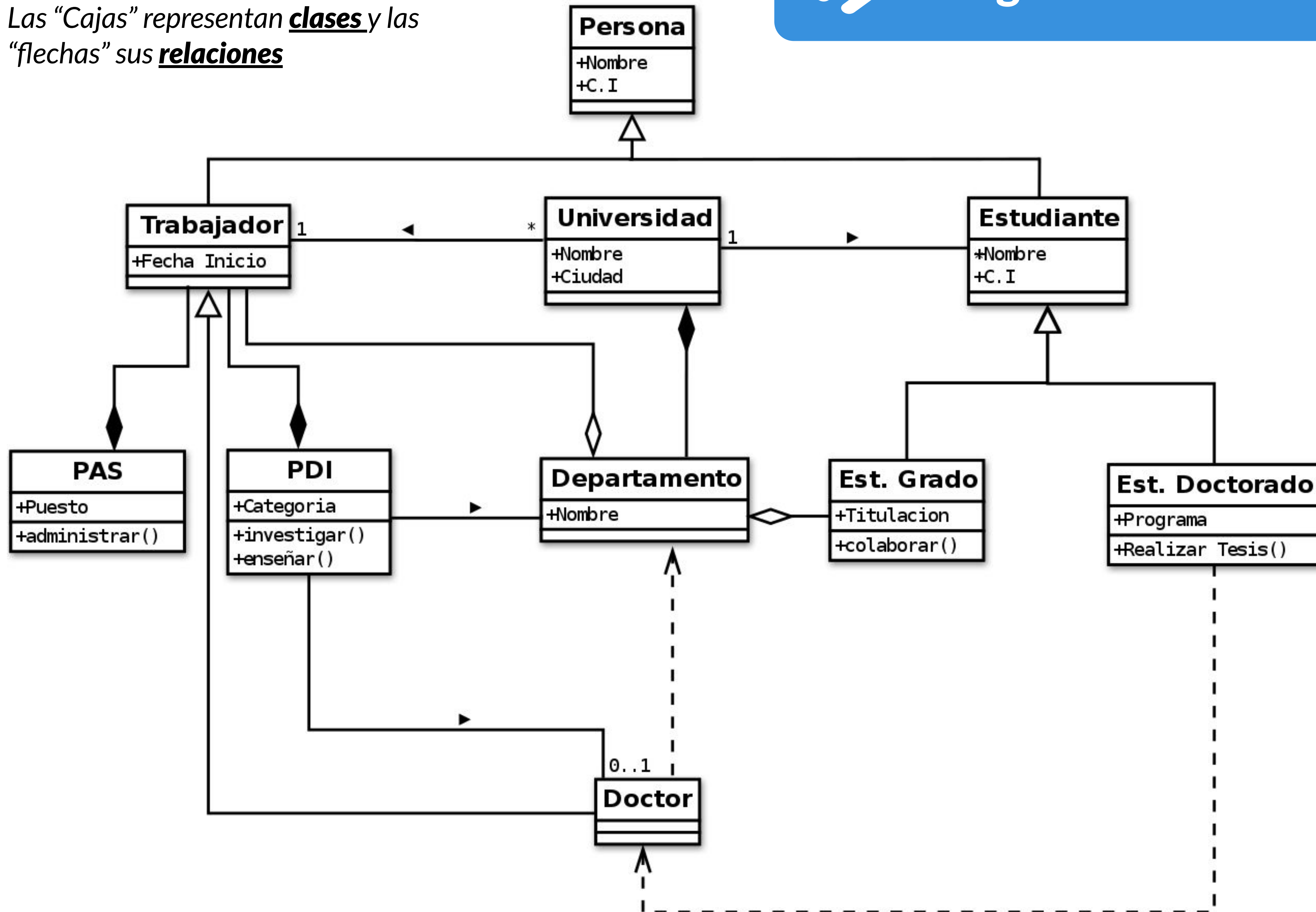
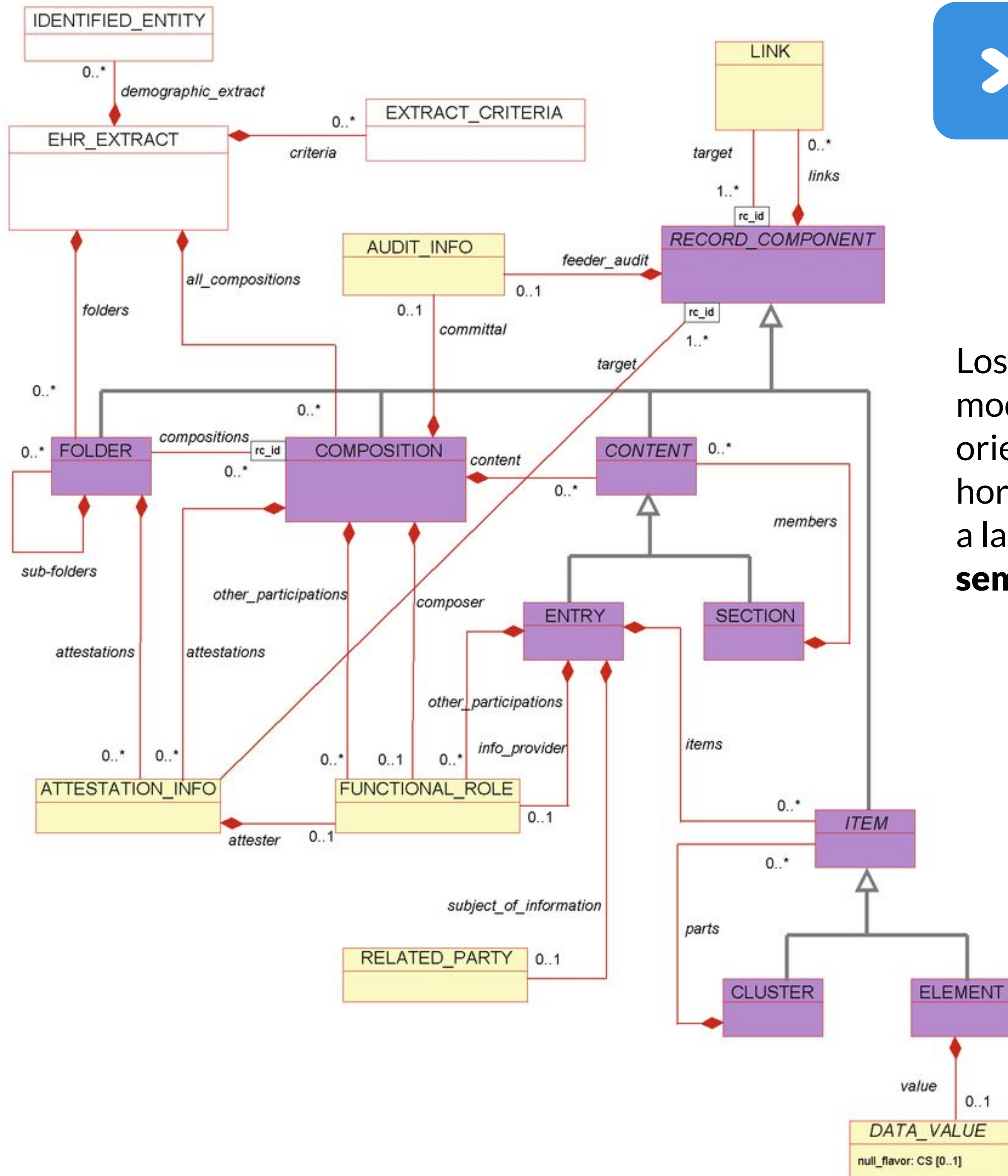
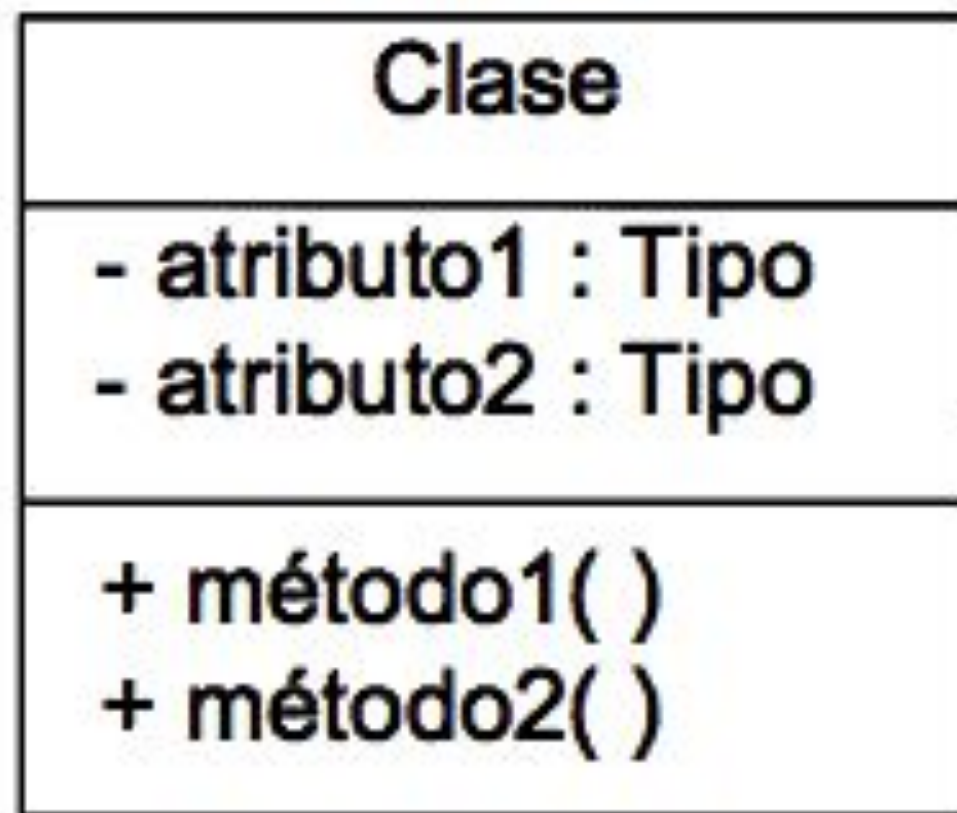


Diagrama de Clases



Los diagramas de clase UML son utilizados no solo para modelar programas codificados con el paradigma de orientación objetos, también pueden resultar de utilidad a la hora de diseñar conceptualmente una bases de datos o incluso a la hora de modelar un **estándar de interoperabilidad semántica de historia clínica (ISO 13606)**.



Una clase se representa como una caja, separada en tres zonas por líneas horizontales.

- ❑ En la zona superior se muestra el nombre de la clase y propiedades generales como el estereotipo. El nombre de la clase aparece centrado.
- ❑ La zona central contiene una lista de atributos, uno en cada línea. La notación utilizada para representarlos es:

visibilidad nombre : tipo = valor-inicial

La visibilidad puede ser pública (+), privada (-) o protegida (#).

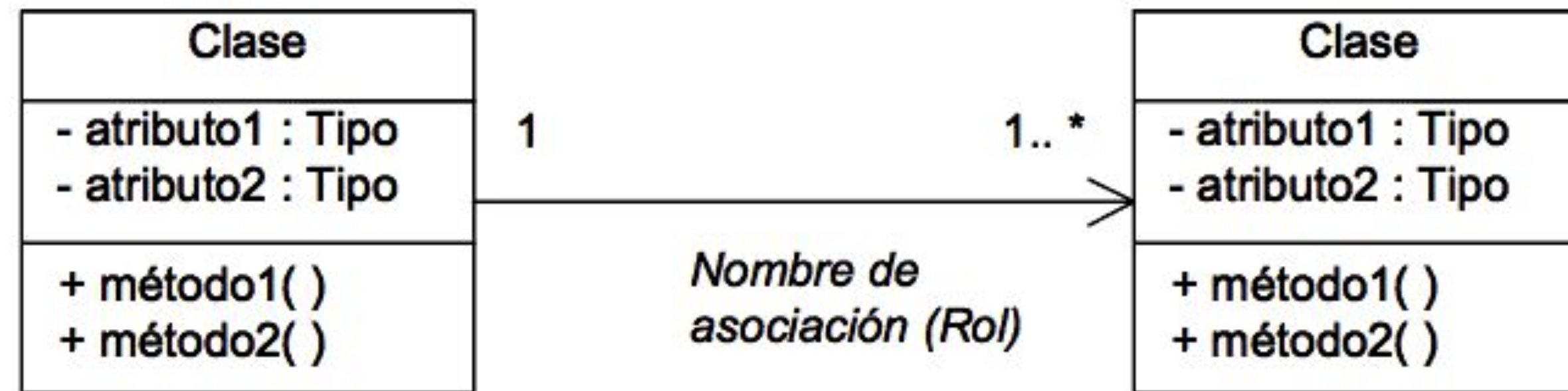
- ❑ En la zona inferior se incluye una lista con las operaciones (métodos o comportamiento) que proporciona la clase. Cada operación aparece en una línea con formato:

visibilidad nombre (lista-de-parámetros): tipo-devuelto

La lista de parámetros es una lista con los parámetros recibidos en la operación separados por comas. El formato de un parámetro es:

nombre : tipo = valor-por-defecto

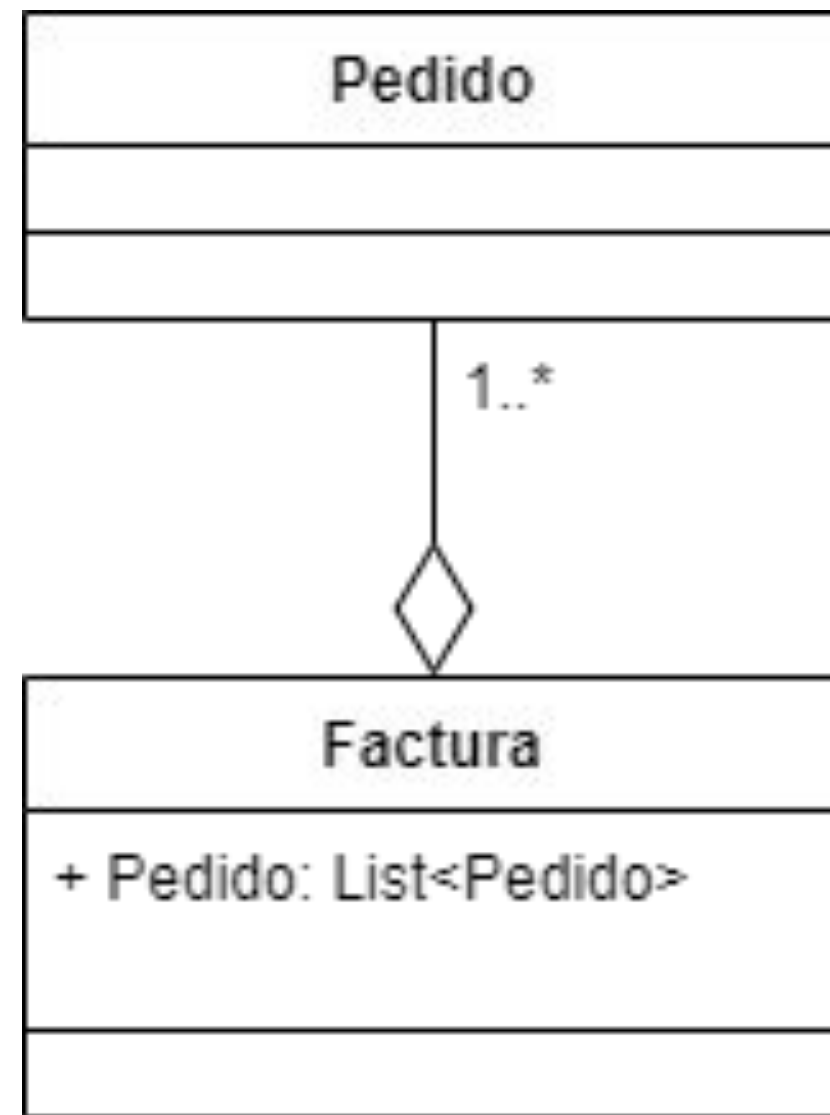
>> Diagrama de Clases. Anatomía



Una relación de asociación se representa como una línea continua entre las clases asociadas. Las diferentes propiedades de la relación se pueden representar con la siguiente notación:

- ❑ Multiplicidad: La multiplicidad puede ser un número concreto, un rango o una colección de números. Por ejemplo un atributo de una clase puede tener una relación uno a uno, uno a muchos o muchos a muchos con otra clase.
- ❑ Navegabilidad: La navegación desde una clase a la otra se representa poniendo una flecha sin relleno en el extremo de la línea, indicando el sentido de la navegación.
- ❑ Rol o nombre de la asociación: Este nombre se coloca junto al extremo de la línea que esta unida a una clase, para expresar cómo esa clase hace uso de la otra clase con la que mantiene la asociación.

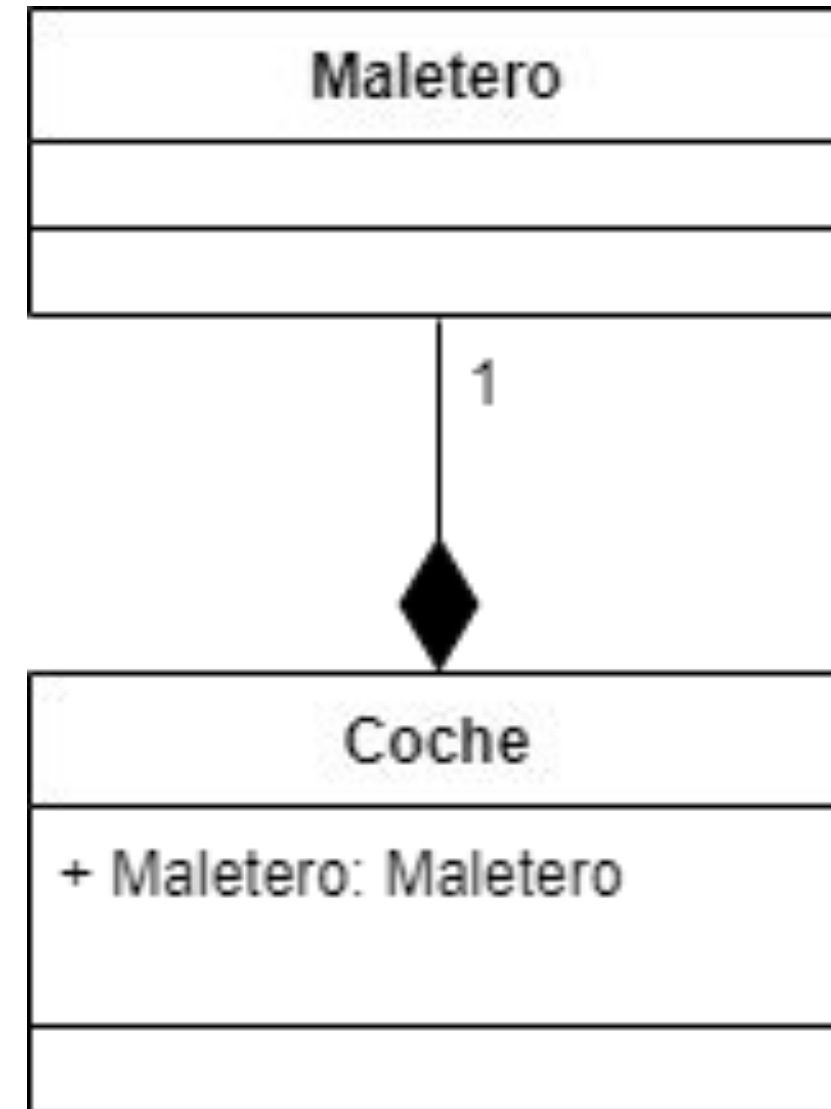
>> Diagrama de Clases. Anatomía



Factura tiene muchos Pedidos

Agregación - “tiene un”
Acoplamiento débil

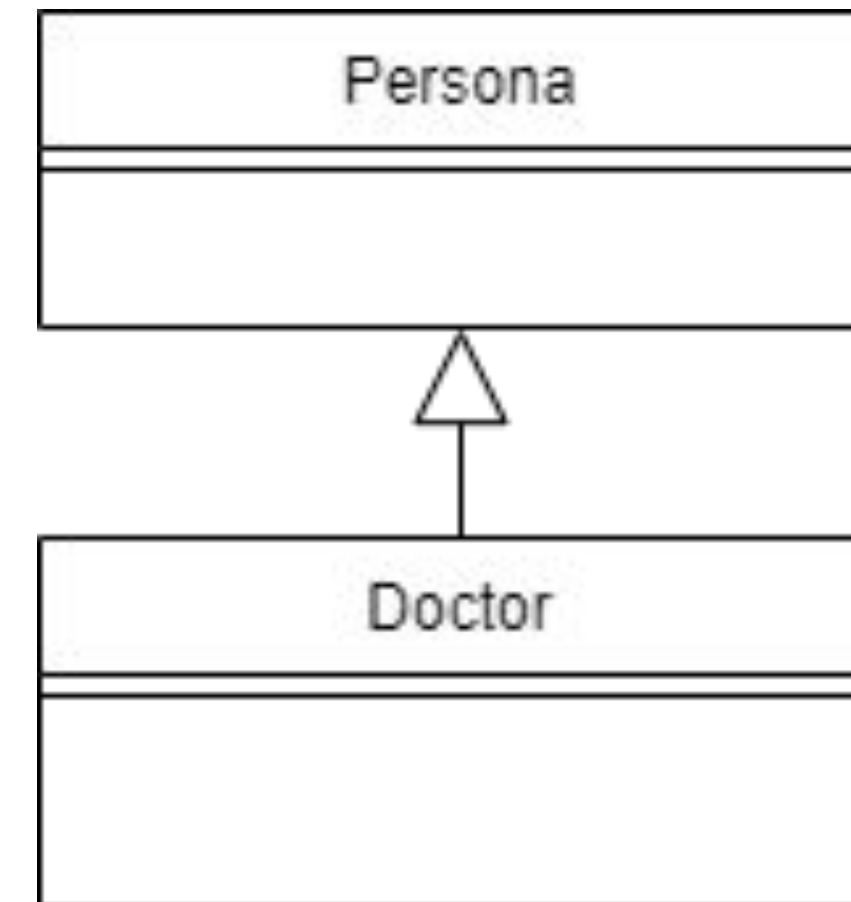
Por ejemplo, la clase “factura” puede tener como atributo una colección de instancias de “pedido”, decimos entonces que la clase “factura” tiene una relación de agregación con la clase “pedido”. Si la factura se destruye los pedidos siguen existiendo



Coche tiene un Maletero

Composición - “tiene un”
Acoplamiento fuerte

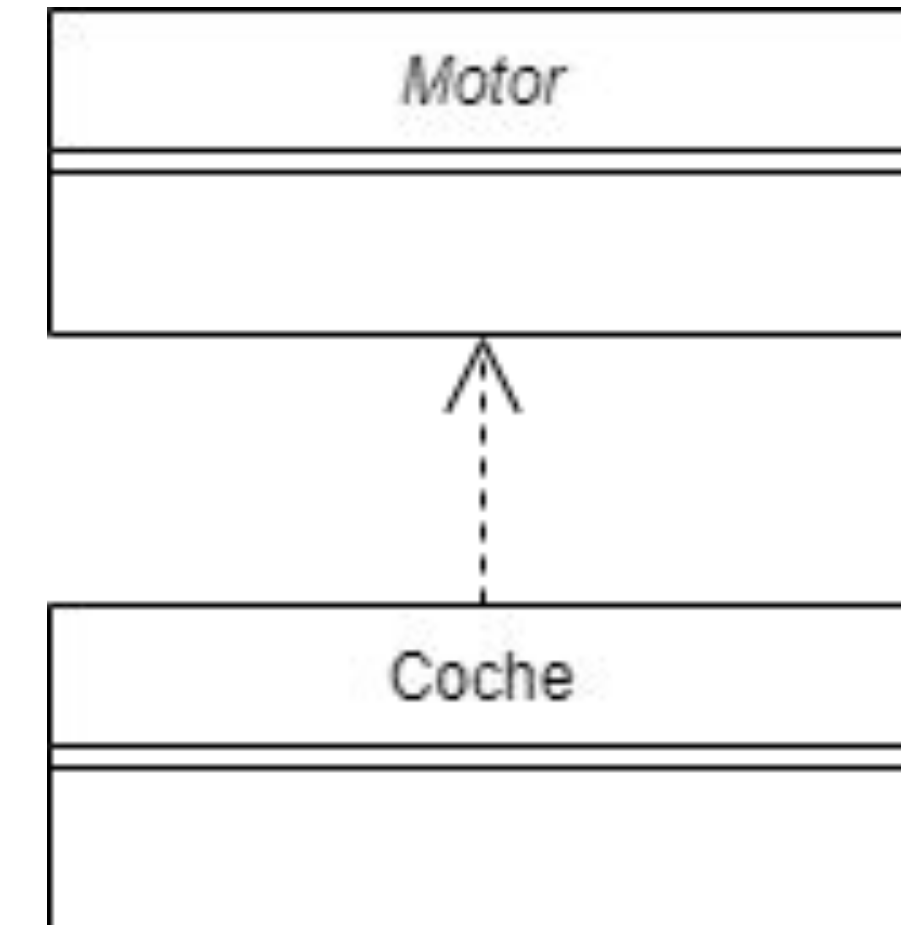
Por ejemplo, la clase “coche” puede tener un atributo de tipo “motor”, decimos entonces que la clase “motor” tiene una relación de composición con “coche” (el motor compone un coche). Si el coche se destruye, el maletero también.



Doctor es una Persona

Herencia - “es un”

Por ejemplo, la clase Paciente y la clase Doctor pueden heredar de la clase Persona.



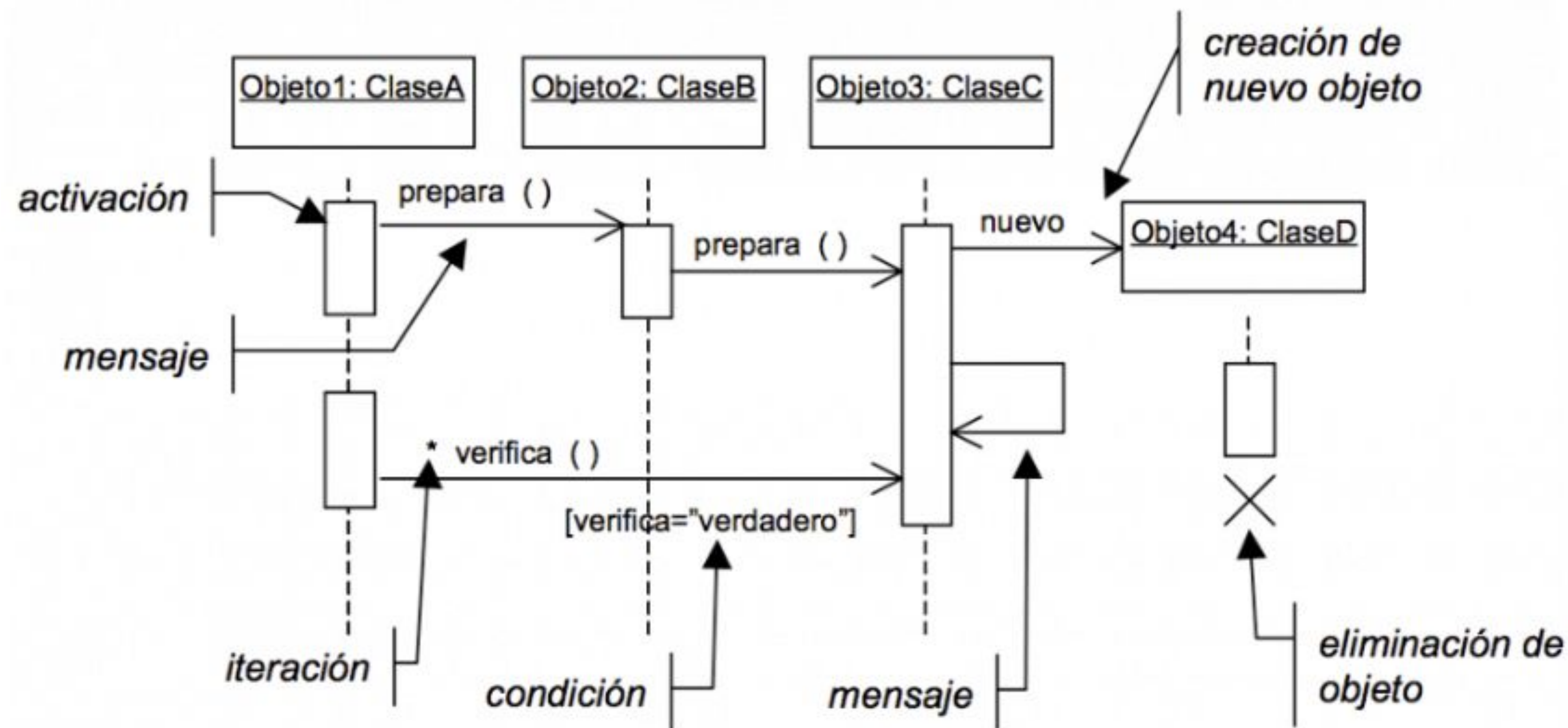
Coche hace uso de Motor

Dependencia - “hace uso de”

Por ejemplo, el método arracar() de la clase coche puede hacer uso de una instancia de la clase “motor”, decimos entonces que la clase “coche” depende de la clase “motor”.

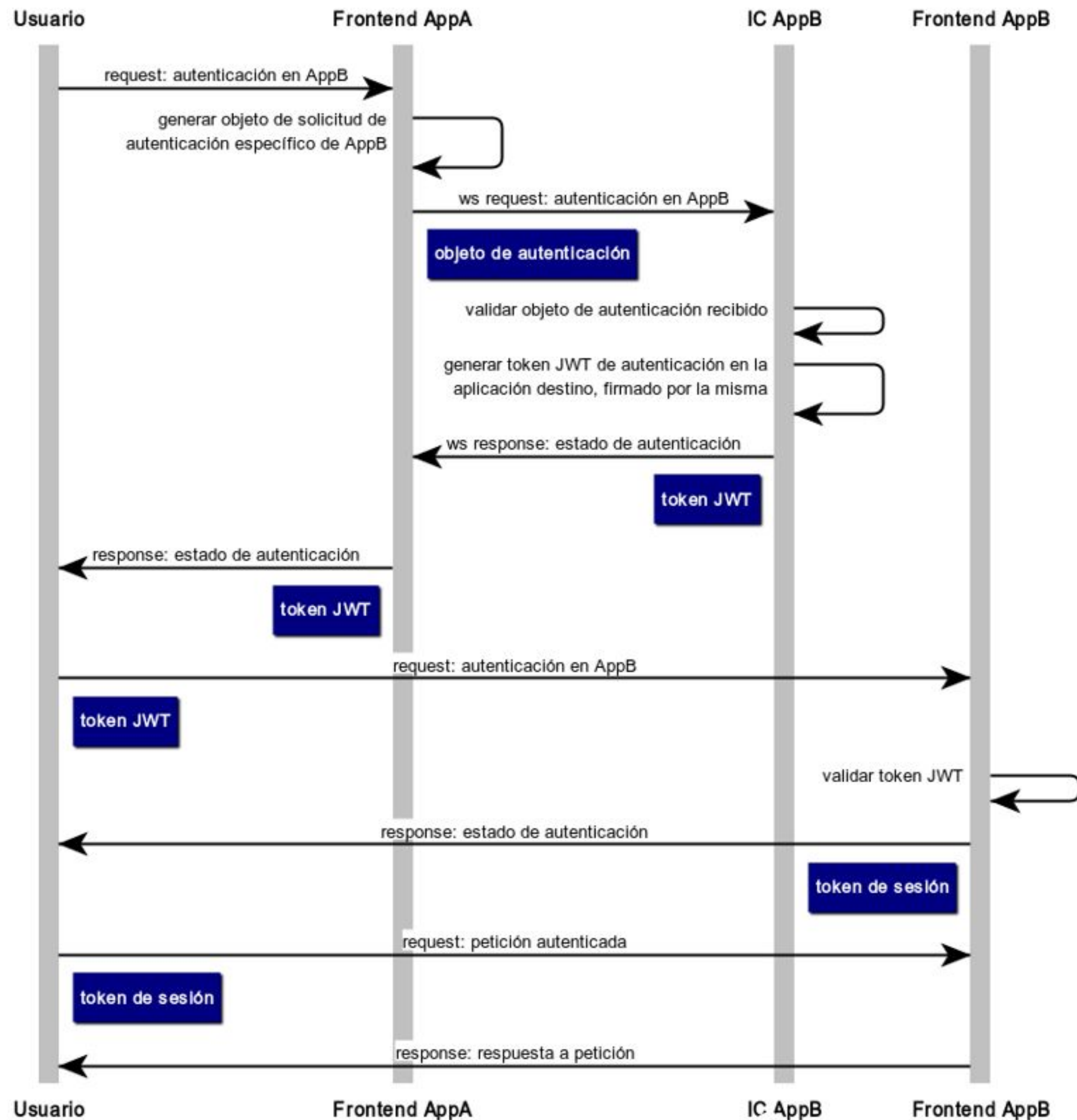
>> Diagrama de Secuencia

Los diagramas de secuencia muestran el flujo de mensajes entre objetos para un determinado caso de uso. Los diagramas de secuencia ponen especial énfasis en el orden y en el momento en que se envían mensajes entre objetos. El propio diagrama explica la secuencia de llamadas que se producen entre los objetos que intervienen. En Programación Orientada a Objetos, un objeto A envía un mensaje M a otro objeto B ejecutando un método del objeto B indicando el mensaje M como argumento (parámetro) del método.



>> Diagrama de Secuencia

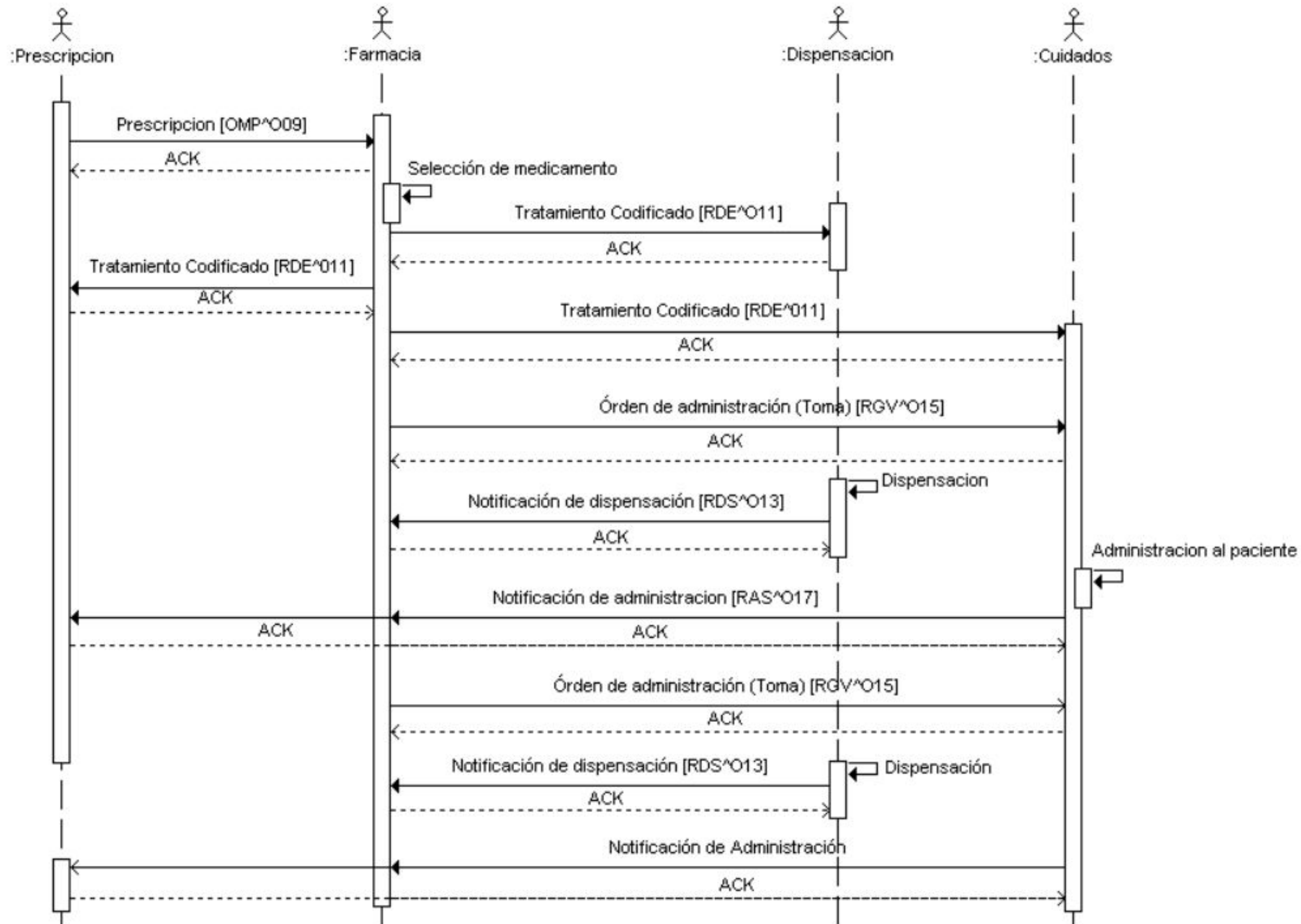
Proceso de autenticación



Representar el tiempo en el eje vertical y flujos de eventos en el eje horizontal el flujo es una forma concisa y útil de comunicación o expresión. Estos diagramas no son sólo útiles para representar relaciones clases y métodos del código sino que también es posible utilizarlo para ayudar a visualizar y documentar procesos de negocio. De esta forma **es posible utilizar el diagrama de secuencia en la fase de análisis.**

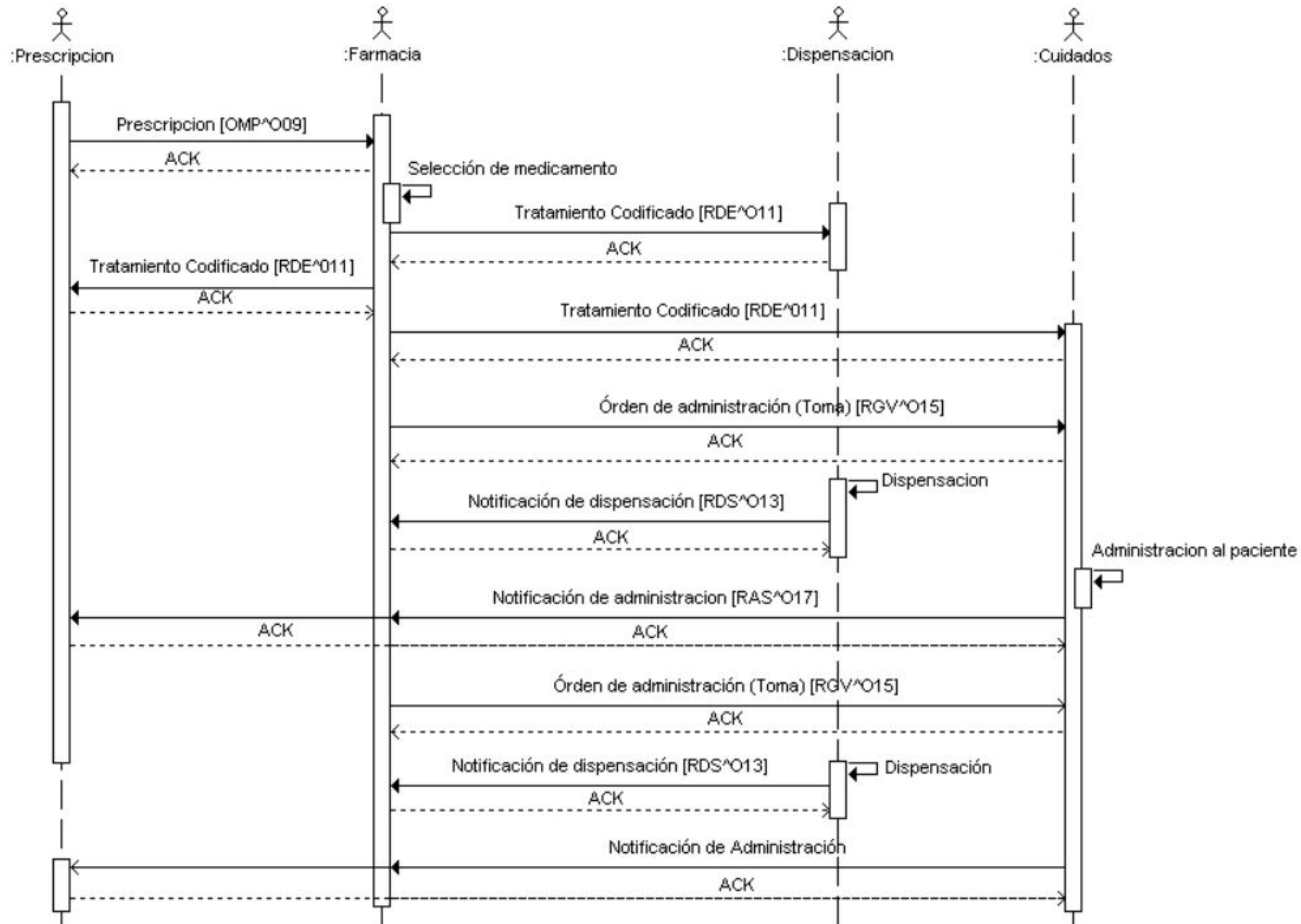
En este Diagrama de Secuencia los ejes verticales no representan objetos sino usuarios y componentes. Este diagrama de secuencia no representa clases y métodos, sino sistemas de información y eventos. Su sencillez consigue transmitir adecuadamente el flujo del proceso.

Diagrama de Secuencia



Nuevamente este diagrama no representa instancias de clases ni invocaciones a métodos. Representa de forma ordenada las comunicaciones que se establecen entre diferentes sistemas de información clínicos y los mensajes que intercambian.

Diagrama de Secuencia



Nuevamente este diagrama no representa instancias de clases ni invocaciones a métodos. Representa de forma ordenada las comunicaciones que se establecen entre diferentes sistemas de información clínicos y los mensajes que intercambian.



¿Qué es el Diseño del software?

El objetivo del diseño del software es concretar y aterrizar cómo se implementarán los componentes y el software especificados en el análisis condicionado por las decisiones arquitectónicas tomadas.

- **Diseño Estructurado**
- **Principios de Diseño**
- **Utilización de diagramas UML**
- **Desarrollo guiado por pruebas**



Desarrollo guiado por pruebas

Las principales características del desarrollo guiado por pruebas son:

- ❑ Se puede prescindir de los “planos” del software, en su lugar el diseño queda documentado en los tests.
- ❑ Exigen pruebas rigurosas para que el diseño “emerja”.

Los planos del software no son los diagramas UML sino el propio código ([Code As Design](#), Jack Reeves, 1992). Los programadores, encargados de transformar los diagramas en código, son sustituidos por el compilador, encargado de transformar el código en binarios (ensamblados). Los programadores, al codificar, diseñan el software, asumiendo el rol de diseñadores.

Una Prueba es un test que el código debe superar. El test define un conjunto de casos de prueba (test case). Un caso de prueba arroja un valor positivo si ante una determinada entrada el código objeto de pruebas devuelve el resultado esperado y un valor negativo si el resultado obtenido es distinto al esperado.

```
public static void main(String[] args) {  
    Alumno alumnoLicenciado = new Alumno("nombre_alumno",5,5);  
  
    if(alumnoLicenciado.SeHaLicenciado()) System.out.println("OK");  
    else System.out.println("NOK");  
  
    Alumno alumnoNoLicenciado = new Alumno("nombre_alumno",5,3);  
  
    if(!alumnoNoLicenciado.SeHaLicenciado()) System.out.println("OK");  
    else System.out.println("NOK");  
}
```

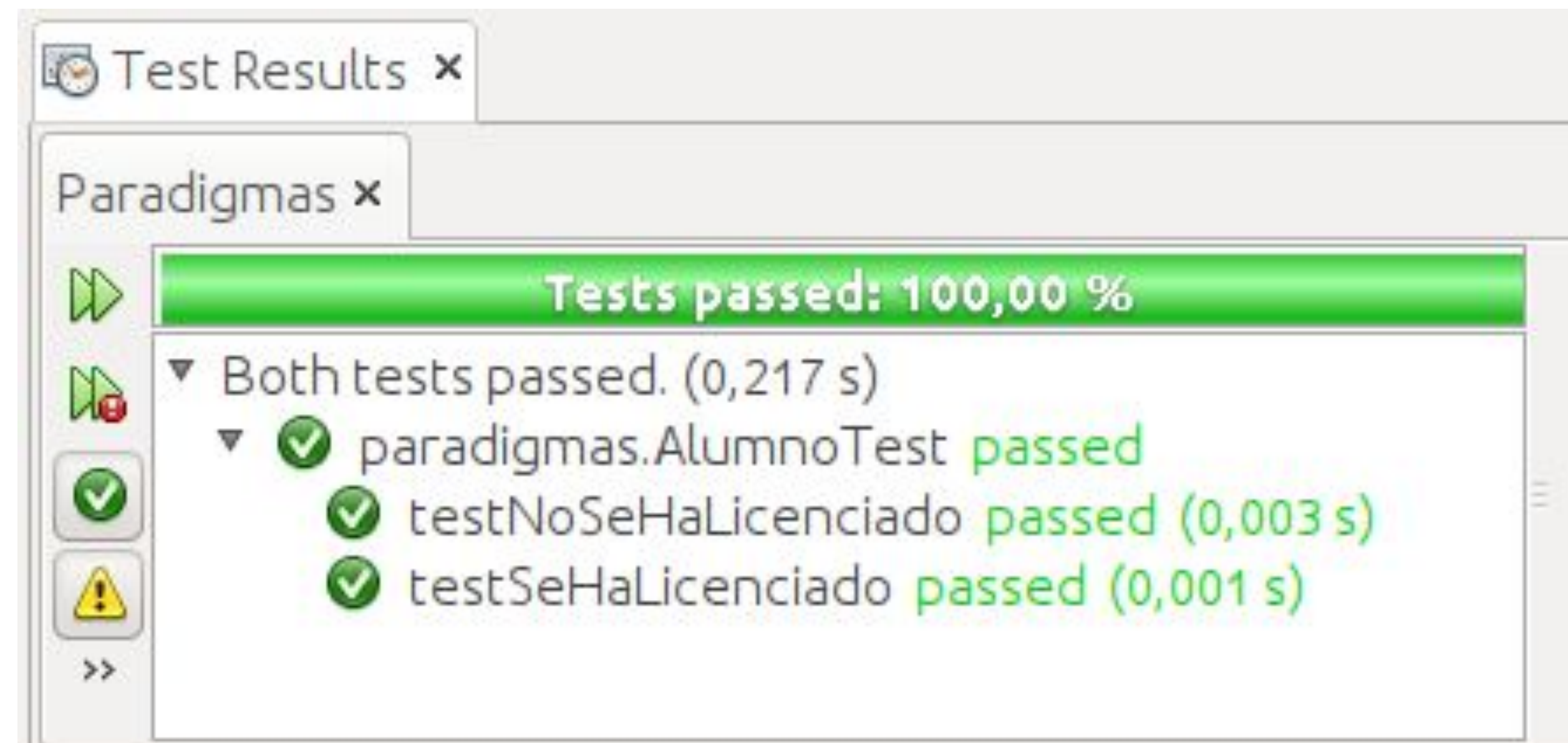

>> Las pruebas ayudan a mejorar el diseño

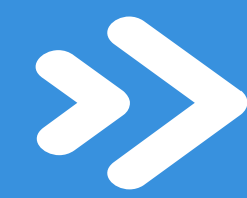



```
@Test
public void testSeHaLicenciado() {
    Alumno alumnoLicenciado = new Alumno("nombre_alumno",5,5);
    assertEquals(true, alumnoLicenciado.SeHaLicenciado());
}

@Test
public void testNoSeHaLicenciado() {
    Alumno alumnoNoLicenciado = new Alumno("nombre_alumno",5,3);
    assertEquals(false, alumnoNoLicenciado.SeHaLicenciado());
}
```

JUnit es un framework de Java que facilita la implementación de pruebas ofreciendo al desarrollador un interface gráfico para ejecutar y consultar el resultado de los tests

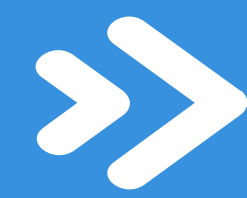




Ventajas de las pruebas

El desarrollo guiado por pruebas no solo hace emerger el diseño de clases, además ofrece importantes ventajas:

- ❑ **Documenta** el código. Los tests reflejan el funcionamiento del código. Es además una documentación que no caduca, es suficiente con ejecutar los tests para asegurar que el código hace exactamente lo que indican las pruebas.
- ❑ **Facilita la refactorización** del código. Refactorizar un código consiste en reescribir un código, haciendo más fácil de entender y extender, sin modificar su comportamiento. El código seguirá pasando los mismos casos de prueba, el código seguirá devolviendo los mismos resultados ante las mismas condiciones de entrada.
- ❑ **Previene las regresiones** del código (bugs). En ocasiones es necesario modificar un segmento de código modificando su comportamiento, añadiendo por ejemplo alguna funcionalidad. Los tests ya escritos serán utilizados por el diseñador/desarrollador para confirmar que la modificación del código no ha roto ninguna de la funcionalidad ya existente.

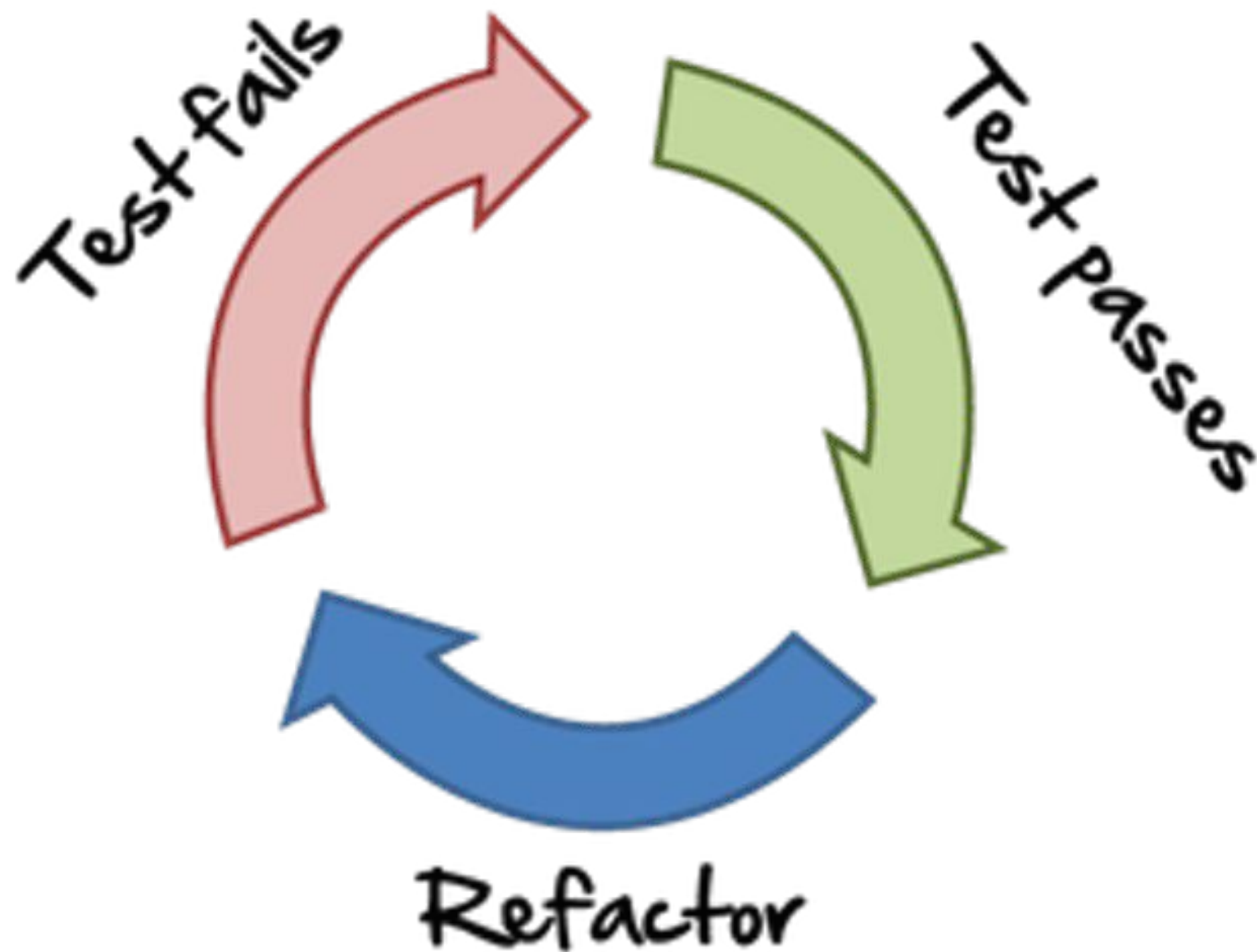


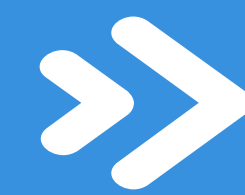
Desarrollo guiado por pruebas

Red. El primer paso es escribir el test. Todavía no se ha escrito la implementación del sistema por lo tanto el test, obviamente arrojará un resultado No OK.

Green. El segundo paso es escribir el código mínimo y necesario para hacer que el test pase. En este punto el resultado del test será OK.

Refactor. El tercer paso es modificar el código limpiándolo y haciéndolo más fácil de entender y mantener. asegurándose que las modificaciones introducidas no provocan fallos en las pruebas escritas.





Chicago vs Londres

En la primera década de los 2000, los practicantes de TDD definieron dos escuelas o estilos: Chicago y Londres.

- El **estilo Chicago** (también denominado “tradicional” o “clásico”) propone comenzar por diseñar primero los objetos del dominio y luego ir agregando más “capas” (presentación, acceso a datos). A este enfoque se le llama “de dentro hacia fuera”.
- El **estilo Londres** propone comenzar primero escribiendo las pruebas de aceptación del usuario (presentación) y posteriormente definir los objetos de dominio a partir de las necesidades de la interface. A este enfoque se le llama “de fuera hacia dentro”.



Scenario: Add two numbers

Given I have entered 50 into the calculator
And I press add
And I have entered 70 into the calculator
When I press equals
Then the result should be 120 on the screen

Scenario Outline: Add three numbers

Given I have entered <ValueA> into the calculator
And I press add
And I have entered <ValueB> into the calculator
And I press add
And I have entered <ValueC> into the calculator
When I press equals
Then the result should be <Result> on the screen

Examples:

ValueA	ValueB	ValueC	Result
50	70	20	140
-10	20	20	30

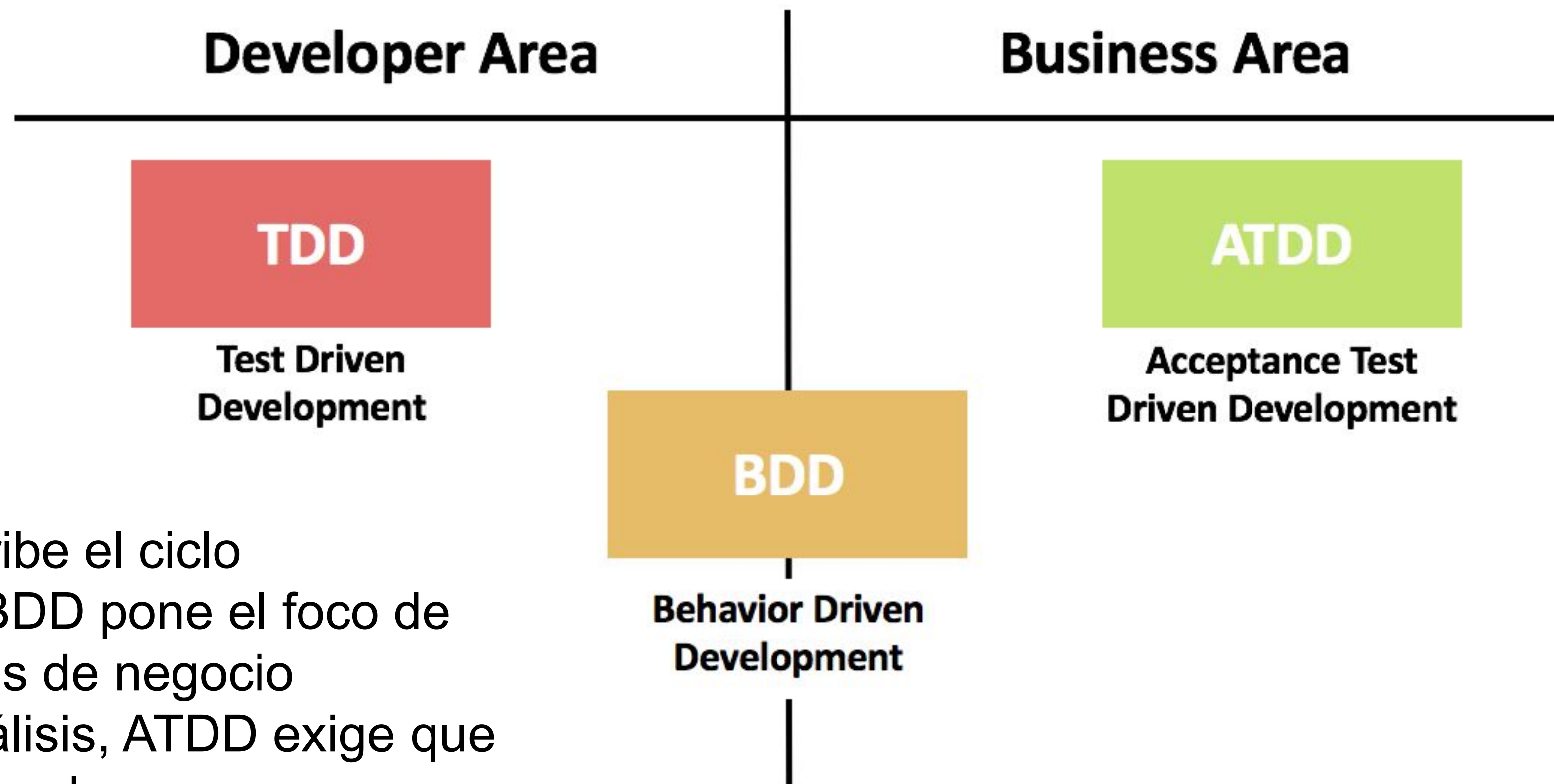
Uno de los inconvenientes de TDD es que la técnica no aclara hasta qué nivel de detalle del código debe cubrirse con tests.

Al igual que TDD también promueve el ciclo Red-Green-Refactor. Si bien TDD no aclara qué tipo de tests deben escribirse, BDD pone el énfasis en los mismos, formalizando su redacción con expresiones Gerkin (Dado ... Cuando ... Entonces ...). Los enunciados de los tests deben extraerse directamente de las especificaciones funcionales indicadas en el análisis. De esta forma BDD da solución a uno de los problemas de TDD.

ATDD



Desarrollo guiado por pruebas



Al igual que TDD prescribe el ciclo Red-Green-Refactor y BDD pone el foco de las pruebas en las reglas de negocio a floradas durante el análisis, ATDD exige que los enunciados de las pruebas sean directamente los criterios de aceptación de las historias de usuario, conectando de esta forma los requisitos, el diseño, la implementación y las pruebas.

TABLA DE CONTENIDOS

01

Arquitectura

02

Diseño Técnico

03

Interface y Experiencia de Usuario (UX/UI)

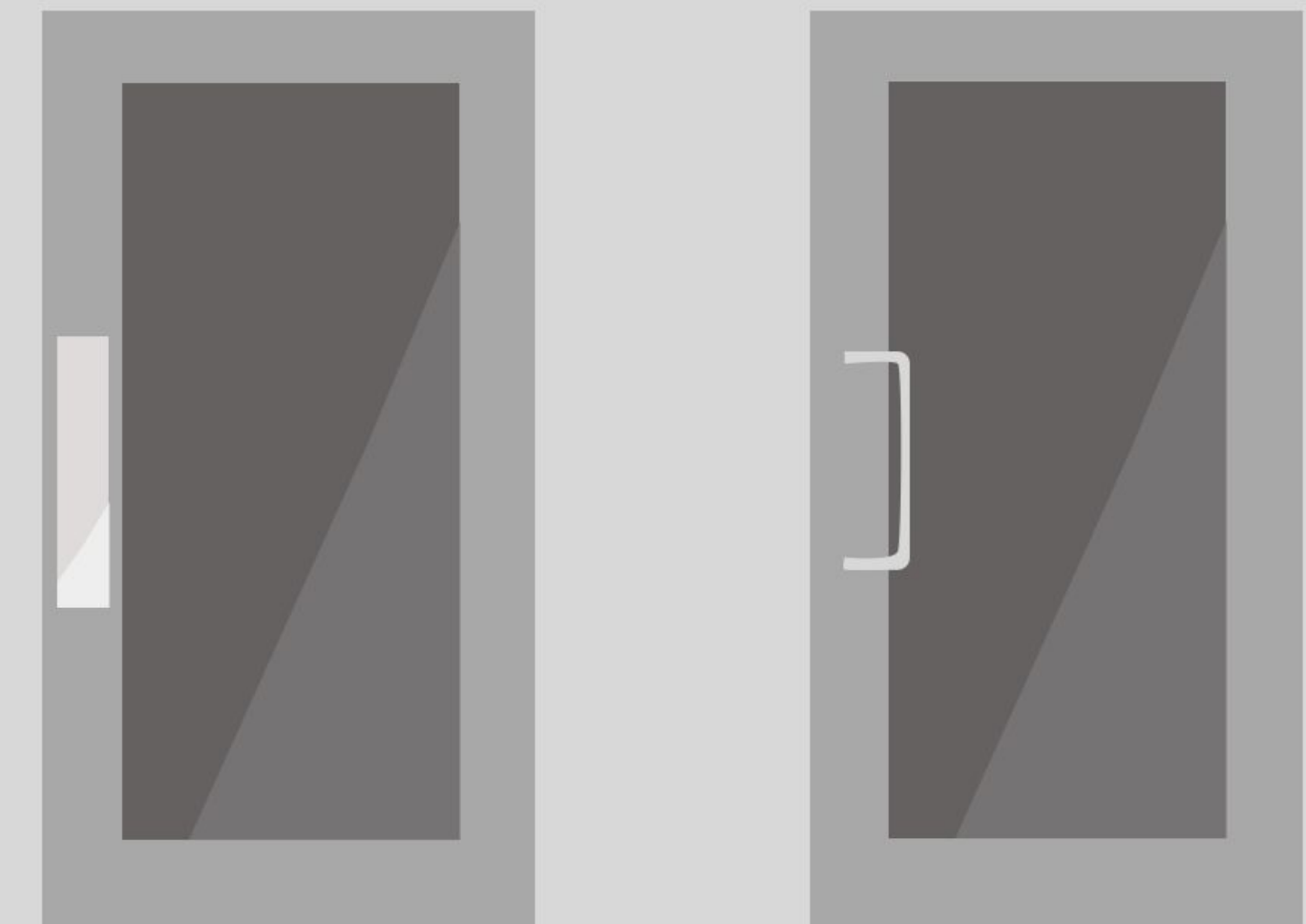
Experiencia e Interface de Usuario

¿Qué es la Experiencia de Usuario?

UX, Experiencia de Usuario, es aquello que una persona percibe al interactuar con un producto o servicio. UX se enfoca en diseñar productos útiles, usables y deseables. Es decir, pone el foco en estudiar cómo y para qué un usuario utiliza un objeto o interactúa con un producto software.

El término User Experience fue acuñado por Donald Norman, un científico social que se unió al equipo de Apple en los 90. En 1988, Norman publicó “The Psychology of Everyday Things” donde establece los pilares de lo que posteriormente se conoció como UX. En este libro Norman utiliza su famosa metáfora las “Puertas de Norman”.

Push or Pull?



Norman's Doors

¿Qué es el Interface de Usuario?

UI (User Interface) hace referencia al aspecto cosmético del producto. El diseño UI se centra en la parte visual. Si UX se encarga de que un producto sea útil para los usuarios, UI lo hace atractivo y visual. El Interfaz del Usuario, es la vista que permite a un usuario interactuar de manera efectiva con un sistema. El UI da el “look & feel” al producto. Los colores, la tipografía, las imágenes son algunos de los elementos con los que trabaja el diseñador UI para hacer que un producto sea atractivo. Pero de nada sirve una cosmética sobresaliente si el producto satisface las necesidades de los usuarios. Por eso, UX y UI deben ir de la mano.



UX vs UI



UX vs UI



1998



2012

UX vs UI

```
A>dir command.com

Volume in drive A is MS-DOS 3_30
Directory of A:\

COMMAND  COM      25276  12-23-90   2:37p
          1 File(s)  254976 bytes free

A>ver

MS-DOS Version 3.30

A>
```



UX vs UI

Una buena forma de entender qué es la experiencia de usuario y qué es el interfaz de usuario es mostrar ejemplos donde la experiencia de usuario sea nula y el interfaz de usuario sea muy pobre. En 2017 un (aburrido) grupo de desarrolladores y diseñadores decidieron organizar un concurso para implementar el peor control de volumen del mundo.

